

# LORA: 대규모 LAN 의 저순위 적응

## 게이지 모델

에드워드 후 예룬 셴 필립 윌리스 제위안 앨런-주  
Yuanzhi Li Shean 왕 루 왕 Weizhu Chen  
마이크로소프트 주식회사  
{에드워드후, 예세, 프랄리스, 제유아나,  
위안질, 스왕, 루, wzchen}@microsoft.com  
yuanzhil@andrew.cmu.edu  
(버전 2)

### 추상적인

자연어 처리의 중요한 패러다임은 일반 도메인 데이터에 대한 대규모 사전 학습과 특정 작업이나 도메인에 대한 적응으로 구성됩니다.

우리는 더 큰 모델을 사전 훈련하고 모든 모델 매개변수를 재훈련하는 전체 미세 조정을 수행합니다. 실현 가능성이 낮아집니다. GPT-3 175B를 예로 들어, 각각 175B 매개변수를 갖는 미세 조정된 모델의 독립 인스턴스를 배포하는 것은 엄청나게 어렵습니다.

비용이 많이 듭니다. 사전 학습된 모델 가치를 고정하고 각 모델에 학습 가능한 순위 분해 행렬을 주입하는 저순위 적응(LoRA)을 제안합니다.

트랜스포머 아키텍처의 계층을 확장하여 다운스트림 작업에 대한 학습 가능한 매개변수 수를 크게 줄였습니다. Adam으로 미세 조정된 GPT-3 175B와 비교했을 때, LoRA는 학습 가능한 매개변수 수를 10,000배까지 줄일 수 있으며 GPU 메모리 요구량이 3배 증가합니다. LoRA는 RoBERTa, DeBERTa, GPT-2, GPT-3에서 모델 품질 면에서 미세 조정과 동등하거나 더 나은 성능을 보입니다. 이는 학습 가능한 매개변수가 적고 학습 처리량이 더 높으며, 어댑터와 달리 추가적인 추론 지연이 없습니다. 또한 다음에 대한 실증적 조사도 제공합니다.

언어 모델 적응의 순위 부족은 효능에 빛을 던집니다.

LoRA. PyTorch와 LoRA의 통합을 용이하게 하는 패키지를 출시합니다.

모델을 제공하고 RoBERTa에 대한 구현 및 모델 검사점을 제공합니다.

DeBERTa 및 GPT-2( <https://github.com/microsoft/LoRA>).

### 1 서론

자연어 처리 분야의 많은 애플리케이션은 하나의 대규모 사전 훈련된 언어 모델을 여러 다운 스트림 애플리케이션에 적용하는 데 의존합니다. 이러한 적용은 일반적으로 미세 조정을 통해 이루어집니다.

사전 학습된 모델의 모든 매개변수를 업데이트합니다. 미세 조정의 가장 큰 단점은 새 모델에 많은 매개변수가 포함된다는 것입니다.

원래 모델과 같은 매개변수. 더 큰 모델이 훈련됨에 따라 몇 달마다 이것은 단순한 "불편함"에서 변화합니다.

GPT-2(Radford et al., b) 또는 RoBERTa large(Liu et al., 2019)에 GPT-3에 대한 중요한 배포 과제(Brown et al., 2020)

1,750억 개의 학습 가능한 매개변수.1

많은 사람들이 일부 매개변수만 조정하여 이를 완화하려고 했습니다.

새로운 작업을 위해 외부 모듈을 학습합니다. 이렇게 하면 각 작업에 대한 사전 학습된 모델에 대해 소수의 작업별 매개변수를 저장하고 로드하여 작업 성능을 크게 향상시킵니다.

배치 시 운영 효율성이 떨어집니다. 그러나 기존 기술은

균등한 기여.

0V1에 비해 이 초안은 더 나은 기준선, GLUE에 대한 실험, 어댑터 지연에 대한 추가 내용을 포함합니다.

1GPT-3 175B는 몇 번의 학습으로 뛰어난 성능을 달성하지만 미세 조정을 통해 성능이 향상됩니다.

부록 A에 표시된 대로 상당히 증가했습니다.

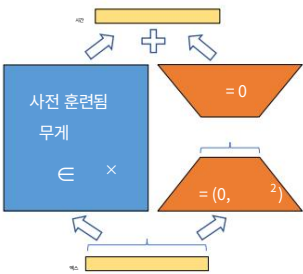


그림 1: 재매개변수화. A와 B만 훈련합니다.

모델 심도를 확장하거나 모델의 사용 가능한 시퀀스 길이를 줄임으로써(Li & Liang, 2021; Lester et al., 2021; Ham-bardzumyan et al., 2020; Liu et al., 2021) 추론 지연 시간(Houlsby et al., 2019; Rebuffi et al., 2017)을 유발하는 경우가 많습니다(3절). 더 중요한 것은, 이러한 방법들이 미세 조정 기준선과 일치하지 않는 경우가 많아 효율성과 모델 품질 간의 상충 관계를 야기한다는 것입니다.

우리는 학습된 과매개변수화된 모델이 실제로 낮은 내재적 차원에 있다는 것을 보여주는 Li et al.(2018a); Aghajanyan et al.(2020)에서 영감을 얻었습니다. 우리는 모델 적응 중 가중치의 변화도 낮은 "내재적 순위"를 가지고 있다고 가정하며, 이를 통해 제안하는 저순위 적응(LoRA) 접근 방식이 도출되었습니다. LoRA를 사용하면 그림 1에서 볼 수 있듯이 사전 학습된 가중치를 고정한 채 적응 중 밀집 계층의 변화에 대한 순위 분해 행렬을 최적화하여 신경망의 일부 밀집 계층을 간접적으로 학습할 수 있습니다. GPT-3 175B를 예로 들면, 전체 순위(즉,  $d$ )가 12,288만큼 높더라도 매우 낮은 순위(즉, 그림 1의  $r$ 은 1 또는 2일 수 있음)가 충분하다는 것을 보여 LoRA는 저장 및 계산 측면에서 효율적입니다.

LoRA는 여러 가지 주요 장점을 가지고 있습니다.

- 사전 학습된 모델을 공유하여 다양한 작업을 위한 여러 개의 소규모 LoRA 모듈을 구축하는 데 사용할 수 있습니다. 그림 1의 행렬 A와 B를 대체하여 공유 모델을 고정하고 작업을 효율적으로 전환할 수 있으며, 이를 통해 저장 공간 요구량과 작업 전환 오버헤드를 크게 줄일 수 있습니다.
- LoRA는 적응형 옵티마이저를 사용할 때 학습 효율을 높이고 하드웨어 진입 장벽을 최대 3배까지 낮춥니다. 대부분의 매개변수에 대해 기울기를 계산하거나 옵티마이저 상태를 유지할 필요가 없기 때문입니다. 대신, 주입된 훨씬 작은 저랭크 행렬만 최적화합니다.
- 간단한 선형 설계를 통해 배포 시 학습 가능한 행렬을 동결된 가중치와 병합할 수 있으며, 구조상 완전히 미세 조정된 모델과 비교했을 때 추론 지연이 발생하지 않습니다.
- LoRA는 많은 기존 방법과 직교하며 다음과 같은 많은 방법과 결합될 수 있습니다. 접두사 튜닝으로. 부록 E에 예시를 제공합니다.

용어 및 규칙 우리는 Transformer 아키텍처를 자주 참조하고 차원에 대한 기존 용어를 사용합니다. Transformer 계층의 입력 및 출력 차원 크기를  $d_{\text{model}}$ 이라고 합니다. 자체 주의 모듈의 쿼리/키/값/출력 투영 행렬을 나타내기 위해  $W_q$ ,  $W_k$ ,  $W_v$  및  $W_o$ 를 사용합니다.  $W$  또는  $W_0$ 는 사전 학습된 가중치 행렬을 나타내고  $\Delta W$ 는 적응 중 누적된 그래디언트 업데이트를 나타냅니다.  $r$ 은 LoRA 모듈의 순위를 나타냅니다.(Vaswani et al., 2017; Brown et al., 2020)에서 정한 규칙을 따르고 모델 최적화를 위해 Adam(Loshchilov & Hutter, 2019; Kingma & Ba, 2017)을 사용하고 Transformer MLP 피드포워드 차원  $d_{\text{ffn}} = 4 \times d_{\text{model}}$ 을 사용합니다.

## 2 문제 진술

본 제안은 학습 목표와는 무관하지만, 동기를 부여하는 사용 사례로 언어 모델링에 중점을 둡니다. 아래는 언어 모델링 문제, 특히 과제별 프롬프트가 주어졌을 때 조건부 확률을 극대화하는 문제에 대한 간략한 설명입니다.

사전 학습된 자기회귀 언어 모델  $P_\Phi(y|x)$ 가  $\Phi$ 에 의해 매개변수화되었다고 가정해 보겠습니다.

예를 들어,  $P_\Phi(y|x)$ 는 GPT(Radford et al., b; Brown et al., 2020)와 같은 일반적인 멀티태스킹 학습기가 될 수 있으며, 이는 Transformer 아키텍처(Vaswani et al., 2017)를 기반으로 합니다. 이 사전 학습된 모델을 요약, 기계 독해(MRC), 자연어 대 SQL(NL2SQL)과 같은 다운스트림 조건부 텍스트 생성 작업에 적용하는 것을 고려해 보세요. 각 다운스트림 작업은 컨텍스트-대상 쌍의 학습 데이터 세트로 표현됩니다.  $Z = \{(x_i, y_i)\}_{i=1, \dots, N}$ , 여기서  $x_i$ 와  $y_i$ 는 모두 토큰 시퀀스입니다. 예를 들어, NL2SQL에서  $x_i$ 는 자연어 쿼리이고  $y_i$ 는 해당 SQL 명령입니다. 요약의 경우  $x_i$ 는 기사의 내용이고  $y_i$ 는 요약입니다.

전체 미세 조정 동안 모델은 사전 학습된 가중치  $\Phi_0$  로 초기화되고 조건부 언어 모델링 목표를 최소화하기 위해 그라디언트를 반복적으로 따라  $\Phi_0 + \Delta \Phi$  로 업데이트됩니다 .

최대  $\Phi$

$$\max_{(x,y) \in \mathcal{Z}} \sum_{t=1}^T \log (P\Phi(y_t|x, y_{<t}))$$

(1)

전체 미세 조정의 주요 단계 중 하나는 각 하위 작업에 대해 차원  $|\Delta \Phi|$ 이  $|\Phi_0|$ 인 서로 다른 매개변수 집합  $\Delta \Phi$ 를 학습한다는 것입니다. 따라서 사전 학습된 모델이 큰 경우(예:  $|\Phi_0|$  이 약 1,750억 개인 GPT-3), 미세 조정된 모델의 여러 독립 인스턴스를 저장하고 배포하는 것은 어려울 수 있으며, 가능하다고 하더라도 어려울 수 있습니다.

본 논문에서는 더욱 매개변수 효율적인 접근법을 채택합니다. 여기서 작업별 매개변수 증가량  $\Delta \Phi = \Delta \Phi(\Theta)$ 는  $|\Theta|$   $|\Phi_0|$ 인 훨씬 작은 크기의 매개변수 집합  $\Theta$ 로 추가로 인코딩됩니다. 따라서  $\Delta \Phi$ 를 구하는 작업은  $\Theta$ 에 대해 최적화하는 작업이 됩니다.

최대  $\Theta$

$$\max_{(x,y) \in \mathcal{Z}} \sum_{t=1}^T \log p\Phi_0 + \Delta \Phi(\Theta)(y_t|x, y_{<t})$$

(2)

이후 섹션에서는 계산 및 메모리 효율 모두 높은 저랭크 표현을 사용하여  $\Delta \Phi$ 를 인코딩하는 방법을 제안합니다. 사전 학습된 모델이 GPT-3 175B일 때, 학습 가능한 매개변수의 수  $|\Theta|$ 는  $|\Phi_0|$ 의 0.01%만큼 작을 수 있습니다 .

3. 기존 솔루션이 충분 하지 않은 가요 ?

우리가 해결하려는 문제는 결코 새로운 것이 아닙니다.전이 학습이 시작된 이래로 수십 개의 연구가 모델 적응을 매개변수 및 계산 측면에서 더 효율적으로 만들려고 노력했습니다.잘 알려진 연구에 대한 개요는 섹션 6을 참조하세요.언어 모델링을 예로 들면, 효율적인 적응과 관련하여 두 가지 주요 전략이 있습니다.어댑터 계층을 추가하는 것(Houlsby et al., 2019; Rebuffi et al., 2017; Pfeiffer et al., 2021; Ruckl e et al., 2020) 또는 입력 계층 활성화의 일부 형태를 최적화하는 것(Li & Liang, 2021; Lester et al., 2021; Hambardzumyan et al., 2020; Liu et al., 2021).그러나 두 전략 모두 한계가 있으며, 특히 대규모이고 지연에 민감한 프로덕션 시나리오에서는 그렇습니다.

어댑터 계층은 추론 지연을 초래합니다 .어댑터에는 많은 변형이 있습니다.우리는 Houlsby et al.(2019)의 원래 설계에 초점을 맞추는데, 이는 Transformer 블록당 두 개의 어댑터 계층을 가지고 있고 Lin et al.(2020)의 최근 설계는 블록당 하나만 있지만 추가 LayerNorm이 있습니다(Ba et al., 2016).계층을 가지지거나 멀티태스킹 설정을 활용하여 전체 지연을 줄일 수 있지만(Ruckl e et al., 2020; Pfeiffer et al., 2021), 어댑터 계층에서 추가 컴퓨팅을 우회하는 직접적인 방법은 없습니다.어댑터 계층은 작은 병목 차원을 갖도록 설계되어 매개변수가 작기 때문에(때로는 원래 모델의 1% 미만) 추가할 수 있는 FLOP가 제한되므로 이는 문제가 되지 않는 것처럼 보입니다.그러나 대규모 신경망은 지연을 낮게 유지하기 위해 하드웨어 병렬 처리에 의존하며 어댑터 계층은 순차적으로 처리해야 합니다. 이는 배치 크기가 일반적으로 1만큼 작은 온라인 추론 설정에서 차이를 만듭니다. 단일 GPU에서 GPT-2(Radford et al., b) 매체로 추론을 실행하는 것과 같이 모델 병렬성이 없는 일반적인 시나리오에서는 병목 현상이 매우 작더라도 어댑터를 사용할 때 지연 시간이 눈에 띄게 증가하는 것을 볼 수 있습니다(표 1).

Shoeybi et al. (2020); Lep-ikhin et al. (2020)에서 수행한 것처럼 모델을 분할해야 할 때 이 문제는 더욱 악화됩니다. 어댑터 매개변수를 여러 번 중복해서 저장하지 않는 한, 추가적인 깊이 AllReduce 및 Broadcast와 같은 동기식 GPU 작업이 필요하기 때문입니다.

프롬프트를 직접 최적화하는 것은 어렵습니다 . 점두사 튜닝(Li & Liang, 2021)에서 예시된 것처럼 다른 방향은 다른 과제에 직면합니다. 본 연구에서는 점두사 튜닝을 최적화하기 어렵고, 그 성능이 훈련 가능한 매개변수에서 비단조적으로 변한다는 것을 관찰했으며, 이는 원 논문에서 관찰된 것과 유사한 결과를 확인시켜 줍니다. 더 근본적으로, 시퀀스 길이의 일부를 적응을 위해 남겨두면 하위 작업을 처리하는 데 사용할 수 있는 시퀀스 길이가 필연적으로 감소하게 되며, 이로 인해 프롬프트 튜닝의 성능이 다른 방법에 비해 저하될 것으로 예상됩니다. 작업 성능에 대한 연구는 5절에서 다루겠습니다.

|                   |                          |                         |                         |
|-------------------|--------------------------|-------------------------|-------------------------|
| 배치 크기             | 32                       |                         | 1                       |
| 시퀀스 길이   $\theta$ | 512                      | 16256                   | 128                     |
|                   | 0.5M                     | 11개월                    | 11개월                    |
| 미세 조정/LoRA        | 1449.4 $\pm$ 0.8         | 338.0 $\pm$ 0.6         | 19.8 $\pm$ 2.7          |
| 어댑터L              | 1482.0 $\pm$ 1.0 (+2.2%) | 354.8 $\pm$ 0.5 (+5.0%) | 23.9 $\pm$ 2.1 (+20.7%) |
| 어댑터H              | 1492.2 $\pm$ 1.0 (+3.0%) | 366.3 $\pm$ 0.5 (+8.4%) | 25.8 $\pm$ 2.2 (+30.3%) |

표 1: GPT-2 매체에서 단일 순방향 패스의 추론 지연 시간(밀리초)으로, 100회 시행 평균. NVIDIA Quadro RTX8000을 사용합니다. "I"는 학습 가능한 데이터 개수를 나타냅니다.

어댑터 계층의 매개변수. AdapterL과 AdapterH는 어댑터 튜닝의 두 가지 변형입니다.

섹션 5.1에서 설명합니다. 어댑터 계층에서 발생하는 추론 지연은 다음과 같은 경우에 상당할 수 있습니다.

온라인, 짧은 시퀀스 길이의 시나리오. 부록 B에서 전체 연구를 확인하세요.

4 우리의 방법

LoRA의 간단한 설계와 실질적인 이점을 설명합니다. 여기에 설명된 원칙들이 적용됩니다.

딥러닝 모델의 모든 고밀도 레이어에 대해, 하지만 우리는 Transformer에서 특정 가중치에만 초점을 맞춥니다.

우리 실험에서는 언어 모델이 동기를 부여하는 사용 사례로 사용되었습니다.

4.1 낮은 순위 매개변수화된 업데이트 행렬

신경망은 행렬 곱셈을 수행하는 여러 개의 밀집된 층으로 구성됩니다. 가중치 이러한 계층의 행렬은 일반적으로 풀 랭크를 갖습니다. 특정 작업에 적응할 때, Aghajanyan et al. (2020)은 사전 훈련된 언어 모델이 낮은 "내재적 차원"을 가지고 있으며 여전히 더 작은 부분 공간에 대한 무작위 투영에도 불구하고 효율적으로 학습합니다. 이를 바탕으로, 적응 과정에서 가중치 업데이트도 낮은 "내재적 순위"를 갖는다는 가설을 세웠습니다. 사전 학습된

가중치 행렬  $W_0 \in \mathbb{R}^{d \times k}$ , 우리는 후자를 낮은 순위의  $d \times k$ 로 표현하여 업데이트를 제한합니다.

구성  $W_0 + \Delta W = W_0 + BA$ , 여기서  $B \in \mathbb{R}^{d \times r}$ ,  $A \in \mathbb{R}^{r \times k}$ , 그리고 순위  $r = \min(d, k)$ .

훈련 중  $W_0$ 는 동결되고 그래디언트 업데이트를 받지 않지만 A와 B에는 훈련 가능한 내용이 포함됩니다.

매개변수.  $W_0$ 와  $\Delta W = BA$ 는 모두 동일한 입력으로 곱해지고 각각의 출력 벡터는 좌표별로 합산됩니다.  $h = W_0x$ 에 대해 수정된 순방향 패스는 다음과 같습니다.

$$h = W_0x + \Delta Wx = W_0x + BAx \tag{3}$$

그림 1에서 재매개변수화를 설명합니다. A에 대해 무작위 가우시안 초기화를 사용합니다.

B의 경우 0이므로  $\Delta W = BA$ 는 훈련 시작 시 0입니다. 그런 다음  $\Delta Wx$ 를  $\alpha$ 로 조정합니다.

r의 상수입니다. Adam으로 최적화할 때  $\alpha$ 를 조정하는 것은 학습을 조정하는 것과 거의 같습니다.

초기화를 적절하게 스케일링하면 속도가 향상됩니다. 결과적으로  $\alpha$ 를 첫 번째 r로 설정합니다.

그리고 조정하지 마세요. 이 스케일링은 우리가 변화할 때 하이퍼파라미터를 재조정할 필요성을 줄이는 데 도움이 됩니다.

r (Yang & 후, 2021).

전체 미세 조정의 일반화. 보다 일반적인 형태의 미세 조정은 다음을 훈련할 수 있도록 합니다.

사전 훈련된 매개변수의 하위 집합입니다. LoRA는 한 걸음 더 나아가 적응 과정에서 전체 랭크를 갖기 위해 가중치 행렬에 대한 누적 그라디언트 업데이트를 요구하지 않습니다. 즉,

모든 가중치 행렬에 LoRA를 적용하고 모든 편향을 훈련합니다. , 우리는 대략적으로 표현력을 회복합니다.

LoRA 순위 r을 사전 훈련된 가중치 행렬의 순위로 설정하여 전체 미세 조정의 효과를 높입니다.

즉, 훈련 가능한 매개변수의 수를 늘리면 LoRA 훈련이 대략적으로 수렴됩니다.

원래 모델을 훈련하는 반면 어댑터 기반 방법은 MLP 및 접두사 기반으로 수렴합니다.

긴 입력 시퀀스를 사용할 수 없는 모델에 대한 방법.

추가 추론 지연 없음. 프로덕션에 배포하면 명시적으로 계산할 수 있습니다.

$W = W_0 + BA$ 를 저장하고 평소처럼 추론을 수행합니다.  $W_0$ 와  $BA$ 가 모두 R에 있다는 점에 유의하세요.

다른 다운스트림 작업으로 전환해야 할 때 BA를 빼서  $W_0$ 를 복구할 수 있습니다.

그런 다음 다른 BA를 추가합니다. , 메모리 오버헤드가 매우 적은 빠른 작업입니다. 중요한 것은

- 2. 가중치에 비해 무시할 수 있는 수의 매개변수를 나타냅니다.
- 3. 힘든 작업에 적응할 때 불가피한 일.

구축을 통해 미세 조정된 모델과 비교했을 때 추론 중에 추가적인 지연이 발생하지 않음을 보장합니다.

#### 4.2 변압기 에 LORA 적용

원칙적으로, 신경망의 가중치 행렬의 모든 부분집합에 LoRA를 적용하여 학습 가능한 매개변수 수를 줄일 수 있습니다. Transformer 아키텍처에서는 셀프 어텐션 모듈에 네 개의 가중치 행렬 ( $W_q, W_k, W_v, W_o$ ) 이 있고 MLP 모듈에는 두 개가 있습니다. 출력 차원이 일반적으로 어텐션 헤드로 분할되지만,  $W_q$  (또는  $W_k, W_v$ ) 를  $d_{model} \times d_{model}$  차원의 단일 행렬로 취급합니다. 연구의 범위를 다운스트림 작업에 대한 어텐션 가중치 조정으로 제한하고, 단순성과 매개변수 효율성을 위해 MLP 모듈을 동결(다운스트림 작업에서 학습되지 않도록)했습니다. 7.1절에서는 Transformer에서 다양한 유형의 어텐션 가중치 행렬 조정에 대한 효과를 추가로 연구합니다. MLP 계층, LayerNorm 계층, 그리고 편향 조정에 대한 실증적 연구는 향후 연구로 미룹니다.

실질적인 이점과 한계. 가장 큰 이점은 메모리 및 저장 공간 사용량 감소입니다. Adam으로 학습된 대용량 트랜스포머의 경우,  $r \cdot d_{model}$  을 사용하면 고정된 매개변수에 대한 최적화 상태를 저장할 필요가 없으므로 VRAM 사용량을 최대 2/3까지 줄일 수 있습니다. GPT-3 175B에서는 학습 중 VRAM 사용량을 1.2TB에서 350GB로 줄였습니다.  $r = 4$ 이고 쿼리 및 값 투영 행렬만 조정하면 체크포인트 크기가 약 10,000배(350GB에서 35MB로) 4 감소합니다.

이를 통해 훨씬 적은 GPU로 학습하고 I/O 병목 현상을 피할 수 있습니다. 또 다른 이점은 모든 매개변수 대신 LoRA 가중치만 교체함으로써 배포 중에도 훨씬 낮은 비용으로 작업 간 전환이 가능하다는 것입니다. 이를 통해 사전 학습된 가중치를 VRAM에 저장하는 머신에서 즉시 교체할 수 있는 다양한 맞춤형 모델을 생성할 수 있습니다. 또한 대부분의 매개변수에 대한 기울기를 계산할 필요가 없기 때문에 GPT-3 175B에서 학습하는 동안 전체 미세 조정5에 비해 25%의 속도 향상을 관찰했습니다.

LoRA에도 한계가 있습니다. 예를 들어, 추가적인 추론 지연 시간을 제거하기 위해 A와 B를 W에 통합하기로 선택하는 경우, 단일 순방향 패스에서 서로 다른 A와 B를 가진 여러 작업에 대한 입력을 일괄 처리하는 것은 간단하지 않습니다. 하지만 지연 시간이 중요하지 않은 시나리오에서는 가중치를 병합하지 않고 일괄 처리에서 샘플에 사용할 LoRA 모듈을 동적으로 선택할 수 있습니다.

#### 5가지 경험적 실험

우리는 GPT-3 175B(Brown et al., 2020)로 확장하기 전에 RoBERTa(Liu et al., 2019), De-BERTa(He et al., 2021), GPT-2(Radford et al., b)에서 LoRA의 다운스트림 작업 성능을 평가합니다. 우리의 실험은 자연어 이해(NLU)에서 생성(NLG)에 이르기까지 광범위한 작업을 포괄합니다. 구체적으로, 우리는 RoBERTa와 DeBERTa에 대해 GLUE(Wang et al., 2019) 벤치마크에서 평가합니다. 우리는 직접 비교를 위해 GPT-2에서 Li & Liang(2021)의 설정을 따르고 GPT-3에서 대규모 실험을 위해 WikiSQL(Zhong et al., 2017)(NL에서 SQL 쿼리)과 SAMSum(Gliwa et al., 2019)(대화 요약)을 추가합니다. 우리가 사용하는 데이터 세트에 대한 자세한 내용은 부록 C를 참조하세요. 모든 실험에는 NVIDIA Tesla V100을 사용합니다.

#### 5.1 기준선

다른 기준선과 광범위하게 비교하기 위해, 이전 연구에서 사용된 설정을 복제하고 가능한 한 보고된 수치를 재사용했습니다. 그러나 이는 일부 기준선이 특정 실험에만 나타날 수 있음을 의미합니다.

미세 조정(FT)은 적응을 위한 일반적인 접근법입니다. 미세 조정 과정에서 모델은 사전 훈련된 가중치와 편향으로 초기화되고, 모든 모델 매개변수는 기울기 업데이트를 거칩니다. 간단한 변형으로는 일부 계층만 업데이트하고 다른 계층은 고정하는 방법이 있습니다. 본 연구에서는 GPT-2에 대한 이전 연구(Li & Liang, 2021)에서 보고된 이러한 기준선 중 하나를 사용하는데, 이 기준선은 마지막 두 계층(FTTop2)만 적응시킵니다.

4배포 중에는 여전히 350GB 모델이 필요하지만 100개의 적응형 모델만 저장하면 됩니다.  $350GB + 35MB \cdot 100 \approx 354GB$ 인 반면  $100 \cdot 350GB \approx 35TB$ 는 그렇지 않습니다.

<sup>5</sup>GPT-3 175B의 경우 전체 미세 조정을 위한 교육 처리량은 V100 GPU당 32.5 토큰/초입니다. 모델 병렬 처리를 위한 가중치 샤드의 수, LoRA의 V100 GPU당 처리량은 43.1 토큰/초입니다.

| 모델 및 방법 # 학습 가능    |          | 매개변수 MNLI SST-2 MRPC CoLA QNLI QQP RTE STS-B 평균 |         |          |          |         |         |          |          |      |      |      |      |
|--------------------|----------|-------------------------------------------------|---------|----------|----------|---------|---------|----------|----------|------|------|------|------|
| 로베이스 (FT)*         | 1억 2500만 | 87.6                                            | 94.8    | 0.1      | 만        | 90.2    | 63.6    | 92.8     | 62.0     | 91.9 | 78.7 | 91.2 | 86.4 |
| RoBbase (비트핏)*     |          | 84.7                                            | 93.7    |          |          | 92.7    | 91.8    |          |          | 84.0 | 81.5 | 90.8 | 85.2 |
| RoBbase (AdptD)*   | 0.3M     | 87.1±.0                                         | 94.2±.1 | 88.5±1.1 | 60.8±.4  | 93.1±.1 | 90.2±.0 | 71.5±2.7 | 89.7±.3  | 84.4 |      |      |      |
| RoBbase (AdptD)*   | 0.9M     | 87.3±.1                                         | 94.7±.3 | 88.4±.1  | 62.6±.9  | 93.0±.2 | 90.6±.0 | 75.9±2.2 | 90.3±.1  | 85.4 |      |      |      |
| RoBbase (LoRA)     | 0.3M     | 87.5±.3                                         | 95.1±.2 | 89.7±.7  | 63.4±1.2 | 93.3±.3 | 90.8±.1 | 86.6±.7  | 91.5±.2  | 87.2 |      |      |      |
| 로블라지 (FT)*         | 355.0M   | 90.2                                            | 96.4    | 68.0     | 94.7     | 86.6    | 92.0    | 88.9     |          | 92.2 |      |      |      |
| 로블라지 (LoRA)        | 0.8M     | 90.6±.2                                         | 96.2±.5 | 90.9±1.2 | 68.2±1.9 | 94.9±.3 | 91.6±.1 | 87.4±2.5 | 92.6±.2  | 89.0 |      |      |      |
| RoBlarge (AdptP) † | 3.0M     | 90.2±.3                                         | 96.1±.3 | 90.2±.7  | 68.3±1.0 | 94.8±.2 | 91.9±.1 | 83.8±2.9 | 92.1±.7  | 88.4 |      |      |      |
| RoBlarge (AdptP) † | 0.8M     | 90.5±.3                                         | 96.6±.2 | 89.7±1.2 | 67.8±2.5 | 94.8±.3 | 91.7±.2 | 80.1±2.9 | 91.9±.4  | 87.9 |      |      |      |
| 로블라그 (AdptH) †     | 6.0M     | 89.9±.5                                         | 96.2±.3 | 88.7±2.9 | 66.5±4.4 | 94.7±.2 | 92.1±.1 | 83.4±1.1 | 91.0±1.7 | 87.8 |      |      |      |
| 로블라그 (AdptH) †     | 0.8M     | 90.3±.3                                         | 96.3±.5 | 87.7±1.7 | 66.3±2.0 | 94.7±.2 | 91.5±.1 | 72.9±2.9 | 91.5±.5  | 86.4 |      |      |      |
| 로블라지 (LoRA) †      | 0.8M     | 90.6±.2                                         | 96.2±.5 | 90.2±1.0 | 68.2±1.9 | 94.8±.3 | 91.6±.2 | 85.2±1.1 | 92.3±.5  | 88.6 |      |      |      |
| DeBXXL (FT)*       | 1500.0M  | 91.8                                            | 97.2    | 93.9     | 92.9     | 91.1    | 92.0    |          | 72.0     | 96.0 | 92.7 |      |      |
| DeBXXL (LoRA)      | 4.7M     | 91.9±.2                                         | 96.9±.2 | 92.6±.6  | 72.4±1.1 | 96.0±.1 | 92.9±.1 | 94.9±.4  | 93.0±.2  | 91.3 |      |      |      |

표 2: RoBERTabase, RoBERTalarge 및 DeBERTaXXL은 다양한 적응 방법을 사용합니다.

GLUE 벤치마크. MNLI, Matthew's의 전체(일치 및 불일치) 정확도를 보고합니다.

CoLA의 경우 상관관계, STS-B의 경우 피어슨 상관관계, 그리고 다른 작업의 경우 정확도. 높을수록 좋습니다.

모든 측정 항목에 대해. \*는 이전 작업에 게시된 숫자를 나타냅니다. †는 설정에서 구성된 실행을 나타냅니다.

공정한 비교를 위해 Houlby et al. (2019)과 유사합니다.

바이어스 전용 또는 BitFit은 다른 모든 것을 고정하고 바이어스 벡터만 훈련하는 기준선입니다.

현재 이 기준선은 BitFit에서도 연구되었습니다(Zaken et al., 2021).

점두사 임베딩 튜닝(PreEmbed)은 입력 토큰 사이에 특수 토큰을 삽입합니다. 이러한 특수 토큰은 학습 가능한 단어 임베딩을 가지며 일반적으로 모델의 어휘에는 포함되지 않습니다. 여기서

이러한 토큰을 배치하는 것은 성능에 영향을 미칠 수 있습니다. 우리는 "점두사"에 중점을 둡니다.

프롬프트에 대한 이러한 토큰과 프롬프트에 추가하는 "중첩"은 모두 Li &에서 논의됩니다.

Liang (2021). lp (또는 li)는 점두사(또는 중위사) 토큰의 개수를 나타냅니다.

학습 가능한 매개변수는  $|\Theta| = d_{\text{model}} \times (lp + li)$ 입니다.

점두사 레이아웃 튜닝(PreLayer)은 점두사 임베딩 튜닝의 확장입니다. 단순히 학습하는 대신

일부 특수한 단어 임베딩(또는 동등하게 임베딩 계층 이후의 활성화)

토큰을 사용하여 모든 트랜스포머 레이어 이후에 활성화 함수를 학습합니다. 이전 레이어에서 계산된 활성화 함수는 학습 가능한 활성화 함수로 대체됩니다. 결과적으로 학습 가능한 매개변수의 개수는 다음과 같습니다.

$|\Theta| = L \times d_{\text{model}} \times (lp + li)$ , 여기서 L은 Transformer 레이어의 수입니다.

Houlby et al. (2019)에서 제안된 어댑터 튜닝은 셀프 어텐션 모듈(및 MLP 모듈)과 후속 잔여 연결 사이에 어댑터 계층을 삽입합니다. 여기에는 두 가지가 있습니다.

어댑터 계층에 바이어스가 있고 그 사이에 비선형성이 있는 완전히 연결된 계층입니다. 이를

원래 설계 AdapterH. 최근 Lin et al.(2020)은 다음과 같은 보다 효율적인 설계를 제안했습니다.

어댑터 계층은 MLP 모듈과 LayerNorm 이후에만 적용됩니다. 이를 AdapterL이라고 합니다. 이는 Pfeiffer et al.(2021)에서 이것 제안한 또 다른 디자인인 AdapterP와 매우 유사합니다. 또한

효율성 향상을 위해 일부 어댑터 계층을 삭제하는 또 다른 기준 호출인 AdapterDrop(Ruckl et al., 2020)을 포함합니다

(AdapterD). 최대 효율을 위해 가능한 한 이전 연구의 수치를 인용합니다.

비교할 기준선의 수는 첫 번째 열에 별표(\*)가 있는 행에 있습니다.

모든 경우에  $|\Theta| = L \times \text{Adpt} \times (2 \times d_{\text{model}} \times r + r + d_{\text{model}}) + 2 \times L \times L_{\text{N}} \times d_{\text{model}}$  여기서 L Adpt

는 어댑터 계층의 수이고 L LN은 훈련 가능한 LayerNorm의 수입니다(예: AdapterL의 경우).

LoRA는 기존 가중치 행렬에 병렬로 훈련 가능한 순위 분해 행렬 쌍을 추가합니다.

4.2절에서 언급했듯이, 우리는 단순성을 위해 대부분의 실험에서 Wq와 Wv에만 LoRA를 적용합니다.

학습 가능한 매개변수의 수는 순위 r과 원래 가중치의 모양에 따라 결정됩니다.

$|\Theta| = 2 \times L \times \text{LoRA} \times d_{\text{model}} \times r$ , 여기서 L LoRA는 LoRA를 적용하는 가중치 행렬의 수입니다.

| 모델 및 방법          | # 훈련 가능  | E2E NLG 챌린지 |          |         |         |         |          |          |
|------------------|----------|-------------|----------|---------|---------|---------|----------|----------|
|                  | 매개변수     | BLEU        | NIST     | MET     | ROUGE-L | CIDEr   |          |          |
| GPT-2 M(피트)*     | 354.92M  | 68.2        | 0.37M    | 8.62    | 46.2    | 71.0    | 2.47     |          |
| GPT-2 M(어댑터L)*   | 66.3     |             |          | 8.41    | 45.0    | 69.8    | 2.40     |          |
| GPT-2 M(어댑터L)*   | 11.09M   | 68.9        | 11.09M   | 8.71    | 46.1    | 71.3    | 2.47     |          |
| GPT-2 M(어댑터H)    | 67.3±.6  | 8.50±.07    | 46.0±.2  | 25.19M  | 68.1    | 0.35M   | 70.7±.2  | 2.44±.01 |
| GPT-2 M(FTTop2)* | 69.7     | 0.35M       | 70.4±.1  | 8.59    | 46.0    | 70.8    | 2.41     |          |
| GPT-2 M(프리레이어)*  | 8.85±.02 | 46.8±.2     | 8.81     | 46.1    | 71.4    | 2.49    |          |          |
| GPT-2 M(LoRA)    |          |             |          |         |         | 71.8±.1 | 2.53±.02 |          |
| GPT-2 L(피트)*     | 774.03M  | 68.5        |          | 8.78    | 46.0    | 69.9    | 2.45     |          |
| GPT-2 L(어댑터L)    | 0.88M    | 69.1±.1     | 8.68±.03 | 46.3±.0 | 23.00M  | 71.4±.2 | 2.49±.0  |          |
| GPT-2 L(어댑터L)    | 68.9±.3  | 8.70±.04    | 46.1±.1  | 0.77M   | 70.3    | 0.77M   | 71.3±.2  | 2.45±.02 |
| GPT-2 L(프리레이어)*  | 70.4±.1  | 8.89±.02    | 8.85     | 46.2    | 71.7    | 2.47    |          |          |
| GPT-2 L(로라)      | 46.8±.2  |             |          |         |         | 72.0±.2 | 2.47±.02 |          |

표 3: E2E NLG에서 다양한 적응 방법을 적용한 GPT-2 중간(M) 및 대형(L)

도전 과제. 모든 지표에서 높을수록 좋습니다. LoRA는 비슷한 수준의 여러 기준보다 우수한 성능을 보입니다.

또는 더 적은 수의 훈련 가능한 매개변수. 우리가 실행한 실험에 대한 신뢰 구간이 표시됩니다. \*는 다음을 나타냅니다.

이전 작품에 발표된 숫자입니다.

## 5.2 로버타 베이스/대형

RoBERTa(Liu et al., 2019)는 BERT(Devlin)에서 원래 제안된 사전 학습 방법을 최적화했습니다.

et al., 2019a) 및 더 많은 훈련 가능한 기능을 도입하지 않고도 후자의 작업 성능을 향상시켰습니다.

매개변수. RoBERTa는 NLP 리더보드에서 훨씬 더 큰 모델에 추월당했습니다.

최근 몇 년 동안 GLUE 벤치마크(Wang et al., 2019)와 같은 벤치마크는 경쟁력을 유지하고 있습니다.

실무자들 사이에서 크기 대비 인기 있는 사전 학습 모델입니다. 사전 학습된 RoBERTa 기반 모델을 사용합니다.

(125M) 및 HuggingFace Transformers 라이브러리(Wolf et al., 2020)의 RoBERTa large(355M)

GLUE의 작업에 대한 다양한 효율적인 적응 접근 방식의 성능을 평가합니다.

벤치마크. 또한 Housley et al.(2019)과 Pfeiffer et al.(2021)을 그들의 기준에 따라 복제합니다.

설정. 공정한 비교를 보장하기 위해 LoRA를 평가하는 방법에 두 가지 중요한 변경 사항을 적용합니다.

어댑터와 비교합니다. 먼저 모든 작업에 동일한 배치 크기를 사용하고 시퀀스 길이를 사용합니다.

어댑터 기준선과 일치하도록 128개 중 두 번째로 모델을 사전 학습된 모델로 초기화합니다.

MRPC, RTE, STS-B는 미세 조정 기준선처럼 이미 MNLI에 적용된 모델이 아닙니다. 실험

Housley et al. (2019)의 이러한 보다 제한적인 설정에 따라 +로 표시됩니다. 결과는 다음과 같습니다.

표 2(상위 3개 섹션)에 제시되어 있습니다. 사용된 하이퍼파라미터에 대한 자세한 내용은 섹션 D.1을 참조하십시오.

## 5.3 데베르타 XXL

DeBERTa(He et al., 2021)는 훨씬 더 큰 규모로 훈련된 BERT의 최신 변형입니다.

GLUE(Wang et al., 2019) 및 Su-perGLUE(Wang et al., 2020)와 같은 벤치마크에서 매우 경쟁력 있는 성능을 보입니다. LoRA가

여전히 완전한

GLUE에서 미세 조정된 DeBERTa XXL(1.5B)의 결과입니다. 표 2(하단)에 그 결과가 제시되어 있습니다.

사용된 하이퍼파라미터에 대한 자세한 내용은 섹션 D.2를 참조하세요.

## 5.4 GPT-2 중형/대형

LoRA가 NLU의 전체 미세 조정에 대한 경쟁력 있는 대안이 될 수 있음을 보여준 만큼, 우리는 다음과 같은 목표를 가지고 있습니다.

GPT-2 중형 및 대형(Radford et al., NLG 모델)에서 LoRA가 여전히 우세한지 답변하십시오.

b). 직접 비교를 위해 Li & Liang(2021)의 설정과 최대한 유사하게 유지했습니다.

공간적 제약으로 인해 이 섹션에서는 E2E NLG 챌린지(표 3)에 대한 결과만 제시합니다.

WebNLG(Gardent et al., 2017) 및 DART(Nan et al., 2020)에 대한 결과는 섹션 F.1을 참조하세요. 우리

섹션 D.3에서 사용된 하이퍼파라미터 목록을 포함합니다.

| 모델&방법        | # 학습 가능한 WikiSQL MNLI-m SAMSum |          |          |                |
|--------------|--------------------------------|----------|----------|----------------|
|              | 매개변수                           | Acc. (%) | Acc. (%) | R1/R2/RL       |
| GPT-3(FT)    | 175,255.8M                     | 73.8     | 89.5     | 52.0/28.0/44.5 |
| GPT-3(비트넷)   | 14.2M                          | 71.3     | 91.0     | 51.3/27.4/43.5 |
| GPT-3(사전 삽입) | 3.2M                           | 63.1     | 88.6     | 48.3/24.2/40.5 |
| GPT-3(프레이어)  | 20.2M                          | 70.1     | 89.5     | 50.8/27.3/43.5 |
| GPT-3(어댑터H)  | 7.1M                           | 71.9     | 89.8     | 53.0/28.9/44.8 |
| GPT-3(어댑터H)  | 40.1M                          | 73.2     | 91.5     | 53.2/29.0/45.1 |
| GPT-3(로라)    | 470만                           | 73.4     | 91.7     | 53.8/29.8/45.9 |
| GPT-3(로라)    | 37.7M                          | 74.0     | 91.6     | 53.4/29.2/45.1 |

표 4: GPT-3 175B에 대한 다양한 적응 방법의 성능. 논리적 형식을 보고합니다. WikiSQL의 검증 정확도, MultiNLI-matched의 검증 정확도 및 Rouge-1/2/L의 검증 정확도 SAMSum. LoRA는 전체 미세 조정을 포함하여 이전 방식보다 더 나은 성능을 보입니다. 결과는 다음과 같습니다. WikiSQL에서는 약 ±0.5%, MNLI-m에서는 약 ±0.1%, SAMSum에서는 약 ±0.5%의 변동이 있습니다. 세 가지 지표에 대해 ±0.2/±0.2/±0.1입니다.

5.5 GPT-3 175B 로 확장

LoRA에 대한 최종 스트레스 테스트로, 1,750억 개의 매개변수를 갖는 GPT-3까지 확장했습니다. 높은 훈련 비용의 경우, 우리는 주어진 작업에 대한 일반적인 표준 편차만 무작위 시드에 따라 보고합니다. 모든 항목에 대해 하나씩 제공하는 것과는 반대입니다. 사용된 하이퍼파라미터에 대한 자세한 내용은 D.4절을 참조하십시오.

표 4에서 볼 수 있듯이 LoRA는 세 가지 데이터 세트 모두에서 미세 조정 기준과 일치하거나 초과합니다. 참고 그림 2에 표시된 것처럼 모든 방법이 더 많은 훈련 가능한 매개변수를 갖는 것으로부터 단조롭게 이익을 얻는 것은 아닙니다. 256개 이상의 특수 토큰을 사용할 때 상당한 성능 저하가 관찰됩니다. 점두사 임베딩 튜닝 또는 점두사 계층 튜닝을 위한 32개 이상의 특수 토큰. 이는 다음을 뒷받침합니다. Li & Liang(2021)에서도 유사한 관찰 결과가 나왔습니다. 이 현상에 대한 철저한 조사가 진행 중이지만 이 연구의 범위를 벗어나지만, 특수 토큰이 더 많을수록 입력 분포가 사전 훈련 데이터 분포에서 더 멀어질 것으로 추정됩니다. 별도로 다음을 조사 합니다. F.3절에서는 저데이터 체제에서 다양한 적응 접근 방식의 성능에 대해 설명합니다.

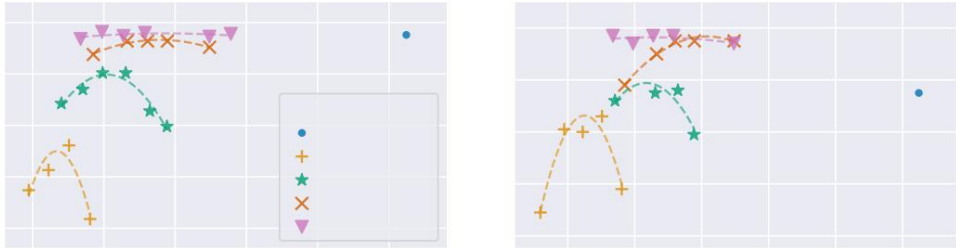


그림 2: GPT-3 175B 검증 정확도 대 여러 적응의 훈련 가능 매개변수 수 WikiSQL과 MNLI 매칭된 방법을 사용합니다. LoRA는 더 나은 확장성과 작업 성능을 보입니다. 표시된 데이터 포인트에 대한 자세한 내용은 섹션 F.2를 참조하세요.

6 관련 작품

Transformer 언어 모델. Transformer(Vaswani et al., 2017)는 시퀀스-투-시퀀스 자기 주의(self-attention)를 많이 사용하는 아키텍처. Radford 등은 (a) Transformer 디코더 스택을 사용하여 자기 회귀 언어 모델링에 이를 적용했습니다. 그 이후로 Transformer 기반 언어 모델은 NLP를 장악하여 많은 작업에서 최첨단 기술을 달성했습니다. 새로운 패러다임이 등장했습니다. BERT(Devlin et al., 2019b) 및 GPT-2(Radford et al., b) - 둘 다 대형 Transformer lan-

대량의 텍스트를 학습한 게이지 모델에서 일반 도메인 데이터에 대한 사전 학습 후 작업별 데이터에 대한 미세 조정을 수행하면 작업별 데이터에 대한 직접 학습보다 상당한 성능 향상을 얻을 수 있습니다. 일반적으로 더 큰 트랜스포머를 학습하면 성능이 향상되며, 이는 여전히 활발한 연구 방향입니다. GPT-3(Brown et al., 2020)는 현재까지 1750억 개의 매개변수로 학습된 가장 큰 규모의 단일 트랜스포머 언어 모델입니다.

프롬프트 엔지니어링 및 미세 조정. GPT-3 175B는 몇 가지 추가 학습 예제만으로도 동작을 조정할 수 있지만, 결과는 입력 프롬프트에 크게 좌우됩니다(Brown et al., 2020). 따라서 원하는 작업에서 모델의 성능을 극대화하기 위해 프롬프트를 구성하고 형식을 지정하는 경험적 기술이 필요한데, 이를 프롬프트 엔지니어링 또는 프롬프트 해킹이라고 합니다.

미세 조정은 일반적인 도메인에서 사전 학습된 모델을 특정 작업에 맞게 재학습하는 것입니다(Devlin et al. (2019b); Radford et al. (a)). 미세 조정의 변형으로는 매개변수의 일부만 학습하는 Devlin et al. (2019b); Collobert & Weston (2008)이 있지만, 실무자들은 종종 다운스트림 성능을 극대화하기 위해 모든 매개변수를 재학습합니다. 그러나 GPT-3 175B의 방대한 용량은 사전 학습과 동일한 메모리 사용량을 가지므로, 생성되는 체크포인트가 크고 하드웨어 진입 장벽이 높아 일반적인 방식으로 미세 조정을 수행하기 어렵게 만듭니다.

매개변수 효율적 적응. 많은 연구자들이 신경망의 기존 계층 사이에 어댑터 계층을 삽입하는 방안을 제안했습니다(Houlsby et al., 2019; Rebuffi et al., 2017; Lin et al., 2020). 본 연구에서는 유사한 병목 구조를 사용하여 가중치 업데이트에 저랭크 제약 조건을 적용합니다. 주요 기능적 차이점은 추론 과정에서 학습된 가중치를 주요 가중치와 병합할 수 있어 지연 시간이 발생하지 않는다는 점인데, 이는 어댑터 계층의 경우와 다릅니다(3절 참조).

어댑터의 대체 확장으로 COMPACTER (Mahabadi et al., 2021)가 있는데, 이는 기본적으로 미리 정해진 가중치 공유 방식을 적용한 크로네커 곱을 사용하여 어댑터 계층을 매개변수화합니다. 마찬가지로, LoRA를 다른 텐서 곱 기반 방법과 결합하면 매개변수 효율성을 잠재적으로 향상시킬 수 있으며, 이는 향후 연구 과제로 남겨둡니다. 최근에는 많은 연구자들이 미세 조정 대신 입력 단어 임베딩을 최적화하는 방안을 제안했는데, 이는 프롬프트 엔지니어링의 연속적이고 미분 가능한 일반화와 유사합니다(Li & Liang, 2021; Lester et al., 2021; Hambardzumyan et al., 2020; Liu et al., 2021). 본 논문의 실험 섹션에서는 Li & Liang(2021)과의 비교를 포함했습니다.

그러나 이 작업 라인은 위치 임베딩이 학습될 때 작업 토큰에 사용 가능한 시퀀스 길이를 차지하는 더 많은 특수 토큰을 프롬프트에 사용하여 확장할 수 있습니다.

딥러닝에서의 저랭크 구조. 저랭크 구조는 머신러닝에서 매우 흔합니다. 많은 머신러닝 문제는 특정한 고유 저랭크 구조를 가지고 있습니다(Li et al., 2016; Cai et al., 2010; Li et al., 2018b; Grasedyck et al., 2013). 더욱이, 많은 딥러닝 과제, 특히 과도하게 매개변수화된 신경망을 사용하는 과제의 경우, 학습된 신경망은 학습 후 저랭크 특성을 갖는 것으로 알려져 있습니다(Oymak et al., 2019). 일부 이전 연구에서는 원래 신경망을 학습할 때 낮은 순위 제약 조건을 명시적으로 적용하기도 했습니다(Sainath et al., 2013; Povey et al., 2018; Zhang et al., 2014; Jaderberg et al., 2014; Zhao et al., 2016; Kho-dak et al., 2021; Denil et al., 2014). 그러나 우리가 아는 한, 이러한 연구 중 어느 것도 다운스트림 작업에 대한 적응을 위해 동결된 모델에 낮은 순위 업데이트를 적용하는 것을 고려하지 않았습니다. 이론 문헌에서는 기본 개념 클래스가 특정 저랭크 구조를 가질 때(Ghorbani et al., 2020; Allen-Zhu & Li, 2019; Allen-Zhu & Li, 2020a), 신경망이 해당 (유한 폭) 신경 텐젠트 커널(Allen-Zhu et al., 2019; Li & Liang, 2018)을 포함한 다른 고전적인 학습 방법보다 성능이 우수한 것으로 알려져 있습니다. Allen-Zhu & Li(2020b)의 또 다른 이론적 결과는 저랭크 적응이 적대적 학습에 유용할 수 있음을 시사합니다. 결론적으로, 본 연구에서 제안하는 저랭크 적응 업데이트는 기존 문헌에서 충분히 도출된 결과라고 생각합니다.

## 7. 저순위 업데이트 이해하기

LoRA의 경험적 이점을 고려하여, 하위 작업에서 학습된 저랭크 적응의 특성을 더욱 자세히 설명하고자 합니다. 저랭크 구조는 하드웨어 진입 장벽을 낮춰 여러 실험을 병렬로 실행할 수 있게 해줄 뿐만 아니라, 업데이트 가중치와 사전 훈련된 가중치의 상관 관계를 더 잘 해석할 수 있도록 합니다. 본 연구는 GPT-3 175B에 집중하여 진행되었으며, 이 경우 작업 성능에 부정적인 영향을 미치지 않으면서 학습 가능한 매개변수를 최대 10,000배까지 감소시켰습니다.

우리는 다음 질문에 답하기 위해 일련의 경험적 연구를 수행합니다. 1) 매개변수 예산 제약이 주어지면 사전 훈련된 Transformer의 어떤 가중치 행렬 하위 집합을 적용해야 하나요?

하루 성능을 극대화하기 위해? 2) "최적" 적응 행렬  $\Delta W$ 는 실제로 순위가 부족한가요? 그렇다면 실제로 사용하기에 적합한 순위는 무엇 인가요? 3)  $\Delta W$ 와  $W$ ?  $\Delta W$ 는  $W$ 와 높은 상관관계가 있나요?  $\Delta W$ 는  $W$ 에 비해 얼마나 큰가요?

우리는 질문 (2) 및 (3)에 대한 답변이 사용의 기본 원칙을 밝혀준다고 믿습니다.  
다운스트림 작업을 위한 사전 훈련된 언어 모델은 NLP에서 중요한 주제입니다.

7.1 LORA를 변압기 의 어떤 가중치 행렬에 적용 해야 합니까 ?

제한된 매개변수 예산이 주어졌을 때 LoRA를 사용하여 어떤 유형의 가중치를 적용해야 합니까?  
다운스트림 작업에서 최고의 성능을 발휘하는 방법은 무엇일까요? 4.2절에서 언급했듯이, 우리는 가중치만 고려합니다.  
자체 주의 모듈의 행렬, 매개변수 예산을 18M(저장 시 약 35MB)으로 설정했습니다.  
GPT-3 175B의 FP16에서, 우리가 한 유형의 주의 가중치를 적용하는 경우  $r=8$ 에 해당합니다.  
96개 층 모두에 대해 두 가지 유형을 적용하면  $r=4$ 가 됩니다. 결과는 표 5에 제시되어 있습니다.

|                         | 학습 가능한 매개변수 수 = 18M |      |      |      |                          |        |
|-------------------------|---------------------|------|------|------|--------------------------|--------|
| 무게 유형<br>랭크 $r$         | Wq                  | Wk   | Wv   | Wo   | Wq, Wk                   | Wv, Wo |
|                         | 8                   | 8    | 8    | 8    | 4                        | 2      |
| WikiSQL ( $\pm 0.5\%$ ) | 70.4                | 70.0 | 73.0 | 73.2 | MultiNLI ( $\pm 0.1\%$ ) | 71.4   |
|                         | 91.0                | 90.8 | 91.0 | 91.3 |                          | 73.7   |
|                         |                     |      |      |      |                          | 73.7   |
|                         |                     |      |      |      |                          | 91.7   |

표 5: 다양한 유형의 WikiSQL 및 MultiNLI에 LoRA를 적용한 후의 검증 정확도  
동일한 수의 학습 가능한 매개변수가 주어졌을 때 GPT-3의 어텐션 가중치,  $W_q$  와  $W_v$ 는 전반적으로 가장 좋은 성능을 제공합니다. 우리는 난수 시드에 대한 표준 편차가 다음과 같다는 것을 발견했습니다.  
첫 번째 열에 보고된 주어진 데이터 세트에 대해 일관성이 있습니다.

모든 매개변수를  $\Delta W_q$  또는  $\Delta W_k$  에 넣으면 성능이 상당히 낮아집니다.  
 $W_q$  와  $W_v$ 를 모두 적용하면 최상의 결과를 얻을 수 있습니다. 이는 4등급이라도  $\Delta W$ 에서 충분한 정보를 포착하므로 더 많은 가중치 행렬을 적용하는 것이 더 바람직합니다.  
더 큰 순위로 단일 유형의 가중치를 조정합니다.

7.2 LORA 의 최적 순위  $r$ 은 무엇입니까 ?

우리는 순위  $r$ 이 모델 성능에 미치는 영향에 주목합니다.  $\{W_q, W_v\}$ 를 다음과 같이 조정합니다.  
 $\{W_q, W_k, W_v, W_o\}$ , 그리고 비교를 위해  $W_q$  만 사용했습니다.

|                          | 체중 유형 | $r=1$ $r=2$ $r=4$ $r=8$ $r=64$ |      |      |      |                |                |
|--------------------------|-------|--------------------------------|------|------|------|----------------|----------------|
|                          |       | Wq                             | Wv   | Wk   | Wo   | Wq, Wv, Wk, Wo | Wq, Wv, Wk, Wo |
| WikiSQL ( $\pm 0.5\%$ )  | Wq    | 69.6                           | 70.5 | 70.4 | 70.0 |                |                |
|                          | Wv    | 73.3                           | 73.7 | 73.8 | 73.5 |                |                |
|                          | Wo    | 73.7                           | 74.0 | 74.0 | 73.9 |                |                |
| MultiNLI ( $\pm 0.1\%$ ) | Wq    | 90.9                           | 91.1 | 90.7 | 90.7 |                |                |
|                          | Wv    | 91.3                           | 91.3 | 91.6 | 91.4 |                |                |
|                          | Wo    | 91.2                           | 91.7 | 91.5 | 91.4 |                |                |

표 6: 순위  $r$ 이 다른 WikiSQL 및 MultiNLI의 검증 정확도, 놀랍게도,  
 $W_q$  만 훈련하는 동안 이러한 데이터 세트에서  $W_q$  와  $W_v$ 를 모두 조정하기 위해서는 1만큼 작은 순위가 충분합니다.  
더 큰  $r$ 이 필요합니다. H.2절에서 GPT-2에 대해 유사한 실험을 수행합니다.

표 6은 놀랍게도 LoRA가 매우 작은  $r$ (더 보기)로 이미 경쟁력 있는 성능을 보인다는 것을 보여줍니다.  
따라서  $\{W_q, W_v\}$  의 경우  $W_q$  만이 아닙니다. ) 이는 업데이트 행렬  $\Delta W$ 가 매우 작을 수 있음을 시사합니다.  
"내재적 순위".6 이 발견을 더욱 뒷받침하기 위해 우리는 학습된 부분 공간의 중복을 확인합니다.  
 $r$ 의 다른 선택과 다른 난수 시드에 의해, 우리는  $r$ 을 증가시키는 것이 다음을 포함하지 않는다고 주장합니다.  
더 의미 있는 부분 공간은 낮은 순위의 적응 행렬이 충분함을 시사합니다.

6 그러나 모든 작업이나 데이터 세트에 작은  $r$ 이 적용될 것이라고 기대하지는 않습니다. 다음과 같은 생각을 고려해 보세요.  
실험: 다운스트림 작업이 사전 학습에 사용된 언어와 다른 언어로 되어 있는 경우 재학습  
전체 모델( $r = d_{\text{model}}$ 인 LoRA와 유사)은 확실히 작은  $r$ 을 사용하는 LoRA보다 성능이 더 좋을 수 있습니다.

서로 다른  $r$  간의 부분 공간 유사성. 동일한 사전 학습된 모델을 사용하여 학습된 적응 행렬  $Ar=8$  과  $Ar=64$ 가 주어졌을 때, 이들은 랭크  $r=8$ 과  $64$ 이며, 특이값 분해를 수행하여 우측 특이 단위 행렬  $UAr=8$  과  $UAr=64$  를 얻습니다.

우리는 다음 질문에 답하고자 합니다.  $UAr=8$  ( $1 \leq i \leq 8$ )의 최상위  $i$ 개 특이 벡터로 구성된 부분 공간 중  $UAr=64$  ( $1 \leq j \leq 64$ )의 최상위  $j$ 개 특이 벡터로 구성된 부분 공간에 얼마나 포함되는가? 우리는 Grassmann 거리를 기반으로 하는 정규화된 부분 공간 유사도를 사용하여 이 값을 측정합니다. (더 자세한 내용은 부록 G 참조)

$$\phi(Ar=8, Ar=64, i, j) = \frac{\|U_{Ar=8}^{(i)} U_{Ar=64}^{(j)}\|_2}{\text{최소}(i, j)} \in [0, 1] \quad (4)$$

에서  $UAr=8$ 은 최상위  $i$ 개의 특이 벡터에 해당하는  $UAr=8$ 의 열을 나타냅니다.  $\phi(\cdot)$ 는  $[0, 1]$ 의 범위를 가지며, 여기서 1은 부분 공간의 완전한 중첩을 나타내고 0은 완전한 분리를 나타냅니다.  $i$ 와  $j$ 를 변화시킬 때  $\phi$ 가

어떻게 변하는지 그림 3을 참조하십시오. 공간 제약으로 인해 96개 중 48번째 층만 살펴볼지만, H.1절에서 볼 수 있듯이 다른 층에도 이 결론이 적용됩니다.

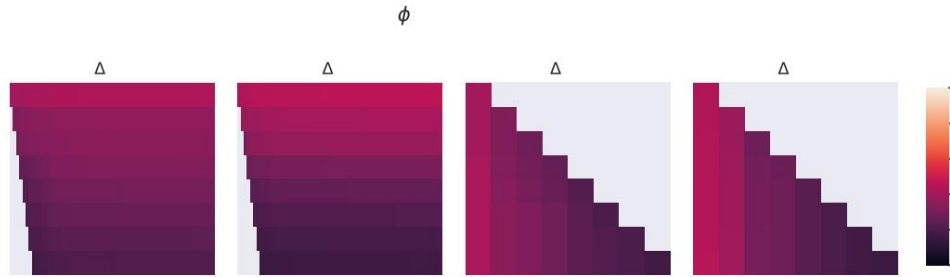


그림 3:  $\Delta W_q$ 와  $\Delta W_v$  모두에 대한  $Ar=8$ 과  $Ar=64$ 의 열 벡터 간의 부분 공간 유사성. 세 번째와 네 번째 그림은 처음 두 그림의 왼쪽 아래 삼각형을 확대한 것입니다.  $r=8$ 의 위쪽 방향은  $r=64$ 에 포함되고, 그 반대의 경우도 마찬가지입니다.

그림 3에서 중요한 관찰 결과를 얻을 수 있습니다.

$Ar=8$ 과  $Ar=64$  사이에서 최상위 특이 벡터에 해당하는 방향은 상당히 겹치지만, 다른 방향은 그렇지 않습니다. 구체적으로,  $Ar=8$ 의  $\Delta W_v$  (또는  $\Delta W_q$ )와  $Ar=64$ 의  $\Delta W_v$  (또는  $\Delta W_q$ )는 정규화 유사도  $> 0.5$ 인 차원 1의 부분 공간을 공유합니다. 이는  $r=1$ 이 GPT-3의 하위 작업에서 매우 우수한 성능을 보이는 이유를 설명합니다.

$Ar=8$ 과  $Ar=64$  모두 동일한 사전 학습된 모델을 사용하여 학습되었기 때문에, 그림 3은  $Ar=8$ 과  $Ar=64$ 의 최상위 특이 벡터 방향이 가장 유용함을 나타내며, 다른 방향들은 학습 과정에서 축적된 무작위 노이즈를 대부분 포함할 가능성이 있습니다. 따라서 적응 행렬은 실제로 매우 낮은 순위를 가질 수 있습니다.

서로 다른 랜덤 시드 간의 부분 공간 유사도. 그림 4에 나타난  $r=64$ 의 두 랜덤 시드 실행 간의 정규화된 부분 공간 유사도를 그래프로 나타내어 이를 더욱 확인합니다.  $\Delta W_q$ 는  $\Delta W_v$ 보다 "내재적 순위"가 더 높은 것으로 보이는데, 이는  $\Delta W_q$ 의 두 실행 모두에서 더 공통적인 특이값 방향이 학습되었기 때문이며, 이는 표 6의 경험적 관찰 결과와 일치합니다.

비교를 위해, 서로 공통된 특이값 방향을 공유하지 않는 두 개의 무작위 가우시안 행렬을 표시합니다.

### 7.3 적응 행렬 $\Delta W$ 는 $W$ 와 어떻게 비교 됩니까?

우리는  $\Delta W$ 와  $W$  사이의 관계를 더 조사합니다. 특히,  $\Delta W$ 가  $W$ 와 높은 상관관계를 가지고 있습니까? (또는 수학적으로  $\Delta W$ 가 대부분  $W$ 의 상위 특이 방향에 포함되어 있습니까?) 또한, 7 유사한 분석이 B와 왼쪽 특이 단위 행렬로 수행될 수 있다는 점에 유의

하십시오.

A는 우리의 실험에 대한 것입니다.

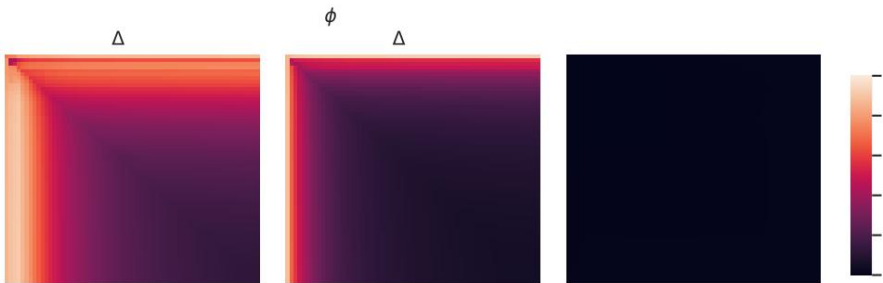


그림 4: 왼쪽 및 가운데: 48번째 층의  $\Delta W_q$  와  $\Delta W_v$  에 대해 두 랜덤 시드에서  $Ar=64$  의 열 벡터 간의 정규화된 부분 공간 유사도 . 오른쪽: 두 랜덤 가우시안 행렬의 열 벡터 간의 동일한 히트맵. 다른 층에 대해서는 H.1절을 참조하십시오.

$\Delta W$ 는  $W$ 의 해당 방향과 비교했을 때 얼마나 "크게" 보일까요? 이는 사전 훈련된 언어 모델을 적용시키는 기본 메커니즘을 밝혀낼 수 있습니다.

이러한 질문에 답하기 위해,  $\Delta W$ 의 좌/우 특이 벡터 행렬을  $U/V$ 로 하여  $UWV$ 를 계산하여  $W$ 를  $\Delta W$ 의  $r$ 차원 부분 공간에 투영합니다. 그런 다음  $UWV$ 와  $W$  사이의 프로베니우스 노름을 비교합니다 .

행렬로 대체하여  $UWV$ 를 계산합니다. 비교를 위해,  $U, V$ 를  $W$ 의 상위  $r$ 개 특이 벡터나 무작위

| $r = 4 \quad r = 64 \quad \Delta W_q \text{ Wq 무작위} \quad \Delta W_q \text{ Wq 무작위} \quad   U \text{ WqV}$ |      |                         |      |
|------------------------------------------------------------------------------------------------------------|------|-------------------------|------|
| $  F = 0.32 \quad 21.67 \quad   Wq  F = 61.95 \quad   \Delta Wq  F = 6.91$                                 |      |                         |      |
|                                                                                                            | 0.02 | 1.90 37.71              | 0.33 |
|                                                                                                            |      | $  \Delta Wq  F = 3.57$ |      |

표 7:  $U \text{ WqV}$ 의 프로베니우스 노름 . 여기서  $U$ 와  $V$ 는 (1)  $\Delta W_q$ , (2)  $W_q$  또는 (3) 랜덤 행렬 의 좌우 상단  $r$  특이 벡터 방향입니다 . 가중치 행렬은 GPT-3의 48번째 계층에서 가져왔습니다.

표 7에서 몇 가지 결론을 도출했습니다. 첫째,  $\Delta W$ 는 무작위 행렬에 비해  $W$ 와 더 강한 상관관계를 가지고 있으며, 이는  $\Delta W$ 가 이미  $W$ 에 있는 일부 특징을 증폭시킨다는 것을 나타냅니다. 둘째,  $\Delta W$ 는  $W$ 의 상위 특이 방향을 반복하는 대신  $W$ 에서 강조되지 않는 방향만 증폭합니다. 셋째, 증폭 계수가 매우 큼니다.  $r = 4$ 일 때  $21.5 \approx 6.91/0.32$ 입니다.

$r = 64$ 일 때 증폭 계수가 더 작은 이유는 H.4절에서 확인할 수 있습니다. 또한,  $W_q$ 에서 상위 특이점 방향을 더 많이 포함함에 따라 상관관계가 어떻게 변하는지에 대한 시각화를 H.3절에서 제공합니다 . 이는 낮은 순위 적응 행렬이 일반적인 사전 학습 모델에서 학습되었지만 강조되지 않은 특정 하위 작업에 대한 중요한 기능을 잠재적으로 증폭시킬 수 있음을 시사합니다.

8 결론 및 향후 연구

방대한 언어 모델을 미세 조정하는 것은 필요한 하드웨어와 다양한 작업에 대한 독립 인스턴스를 호스팅하는 데 드는 저장/스위칭 비용 측면에서 엄청난 비용이 발생합니다. 본 연구에서는 추론 지연 시간을 발생시키지 않고 입력 시퀀스 길이를 줄이지 않으면서도 높은 모델 품질을 유지하는 효율적인 적응 전략인 LoRA를 제안합니다. 중요한 점은 서비스로 배포될 때 대부분의 모델 매개변수를 공유하여 빠른 작업 전환을 가능하게 한다는 것입니다. 본 연구에서는 Transformer 언어 모델에 초점을 맞추었지만, 제안된 원리는 고밀도 계층을 가진 모든 신경망에 일반적으로 적용 가능합니다.

향후 연구 방향은 여러 가지가 있습니다. 1) LoRA는 다른 효율적인 적응 방법과 결합하여 직교적 개선을 제공할 수 있습니다. 2) 미세 조정 또는 LoRA의 메커니즘은 아직 명확하지 않습니다. 사전 학습 과정에서 학습된 특징이 어떻게 변화되어 후속 작업에서 좋은 성능을 발휘하는 것일까요? LoRA는 전체 미세 조정보다 이 질문에 대한 답을 찾기 더 쉽다고 생각합니다.

튜닝. 3) LoRA를 적용할 가중치 행렬을 선택하는 데 주로 휴리스틱에 의존합니다. 더 원칙적인 방법이 있을까요? 4) 마지막으로,  $\Delta W$ 의 랭크 부족은  $W$  역시 랭크 부족일 수 있음을 시사하며, 이는 향후 연구에 영감을 줄 수 있습니다.

## 참고문헌

아르멘 아가자난, 루크 제틀모이어, 소날 굽타. 내재적 차원성이 언어 모델 미세 조정의 효과를 설명한다. arXiv:2012.13255 [cs], 2020년 12월.  
URL <http://arxiv.org/abs/2012.13255>.

Zeyuan Allen-Zhu와 Yuanzhi Li. ResNet은 커널을 넘어 효율적으로 무엇을 학습할 수 있을까? NeurIPS, 2019. 전체 버전은 <http://arxiv.org/abs/1905.10337>에서 확인할 수 있습니다.

Zeyuan Allen-Zhu 및 Yuanzhi Li. 역방향 특징 수정: 딥 러닝의 딥러닝 성능 학습. arXiv 사전 인쇄본 arXiv:2001.04413, 2020a.

Zeyuan Allen-Zhu 및 Yuanzhi Li. 특징 정제: 적대적 훈련이 어떻게 강력하게 수행되는지 딥러닝. arXiv 사전 인쇄본 arXiv:2005.10190, 2020b.

Zeyuan Allen-Zhu, Yuanzhi Li, Zhao Song. 과모수화를 통한 딥러닝을 위한 수렴 이론. ICML, 2019. 전체 버전은 <http://arxiv.org/abs/1811.03962>에서 확인할 수 있습니다.

지미 레이 바, 제이미 라이언 키로스, 제프리 E. 힌튼. 계층 정규화, 2016.

통 B. 브라운, 벤자민 맨, 닉 라이더, 멜라니 수비아, 자레드 카플란, 프라폴라 다리알, 아르빈드 닐라칸탄, 프라나브 샬, 기라쉬 사스트리, 아만다 아스켈, 산디니 아가왈, 아리엘 허버트-보스, 그레첸 크루거, 톰 헤니한, 리원 차일드, 아디티야 라메쉬, 다니엘 M.

지글러, 제프리 우, 클레멘스 윈터, 크리스토퍼 헤세, 마크 첸, 에릭 시글러, 마테우시 리트윈, 스콧 그레이, 벤저민 체스, 잭 클라크, 크리스토퍼 버너, 샘 맥캔들리시, 알렉 래드포드, 일리아 수츠케버, 다리오 아모데이. 언어 모델은 단시간 학습자이다. arXiv:2005.14165 [cs], 2020년 7월. URL <http://arxiv.org/abs/2005.14165>.

Jian-Feng Cai, Emmanuel J Candes, Zuowei Shen. 특이값 임계값 알고리즘 행렬 완성. SIAM 최적화 저널, 20(4):1956–1982, 2010.

Daniel Cer, Mona Diab, Eneko Agirre, Inigo Lopez-Gazpio, Lucia Specia. Semeval-2017 과제 1: 다국어 및 교차언어 중심 의미 텍스트 유사성 평가. 제11회 국제 의미 평가 워크숍(SemEval-2017) 논문집, 2017. doi: 10.18653/v1/s17-2001. URL <http://dx.doi.org/10.18653/v1/S17-2001>.

로난 콜로버트와 제이슨 웨스턴. 자연어 처리를 위한 통합 아키텍처: 멀티태스크 학습을 갖춘 심층 신경망. 제25회 국제 기계학습 학술 대회 논문집, ICML '08, pp. 160–167, 뉴욕, 미국, 2008년 7월. 미국 컴퓨터학회(Association for Computing Machinery). ISBN 978-1-60558-205-4. doi: 10.1145/1390156.1390177. URL <https://doi.org/10.1145/1390156.1390177>.

미샤 데닐, 바박 샤키비, 로랑 딘, 마르크 아우렐리오 란자토, 난도 드 프레이타스. 예측 딥러닝의 매개변수, 2014년.

Jacob Devlin, Ming-Wei Chang, Kenton Lee, Kristina Toutanova. Bert: 언어 이해를 위한 심층 양방향 변환기의 사전 학습, 2019a.

Jacob Devlin, Ming-Wei Chang, Kenton Lee, Kristina Toutanova. BERT: 언어 이해를 위한 딥 양방향 변환기 사전 학습. arXiv:1810.04805 [cs], 2019년 5월 b.  
URL <http://arxiv.org/abs/1810.04805>. arXiv: 1810.04805.

윌리엄 B. 돌란과 크리스 브로켓. 문장 의역 코퍼스 자동 구축. 2005년 제3회 국제의역워크숍(IWP2005) 회의록. URL <https://aclanthology.org/I05-5002>.

Claire Gardent, Anastasia Shimorina, Shashi Narayan, Laura Perez-Beltrachini. 웹 언어 생성 과제: RDF 데이터에서 텍스트 생성. 제 10회 자연어 생성 국제 학술대회 논문집, 124–133쪽, 2017.

베르투스 고르바니, 송 메이, 테오도르 미시아키에비츠, 안드레아 몬타나리. 신경계가

네트워크가 커널 방식보다 성능이 더 좋은가? arXiv 사전 인쇄본 arXiv:2006.13409, 2020.

Bogdan Gliwa, Iwona Mochol, Maciej Biesek 및 Aleksander Wawer. Samsun corpus: 추상적인 요약물 위해 사람이 주석을 추가한 대화 데이터셋입니다. CoRR, abs/1911.12237, 2019. URL <http://arxiv.org/abs/1911.12237>.

Lars Grasedyck, Daniel Kressner, Christine Tobler. 저랭크 텐서에 대한 문헌 조사 근사 기술. GAMM-Mitteilungen, 36(1):53–78, 2013.

함지훈, 다니엘 D. 리. Grassmann 판별 분석: 부분공간 기반 학습에 대한 통합적 관점. ICML, 376–383쪽, 2008. URL <https://doi.org/10.1145/1390156.1390204>.

Karen Hambardzumyan, Hrant Khachatrian, Jonathan May. WARP: 단어 수준 적대적 재프로그래밍. arXiv:2101.00121 [cs], 2020년 12월. URL <http://arxiv.org/abs/2101.00121>. arXiv: 2101.00121.

Pengcheng He, Xiaodong Liu, Jianfeng Gao 및 Weizhu Chen. Deberta: 디코딩 강화 bert 얹매이지 않은 주의로, 2021년.

Neil Houlsby, Andrei Giurgiu, Stanislaw Jastrzebski, Bruna Morrone, Quentin de Laroussilhe, Andrea Gesmundo, Mona Attariyan 및 Sylvain Gelly. NLP를 위한 매개변수 효율적인 전이 학습. arXiv:1902.00751 [cs, stat], 2019년 6월. URL <http://arxiv.org/abs/1902.00751>.

맥스 자더버그, 안드레아 베달디, 앤드류 지서먼. 합성곱 신경망 속도 항상 낮은 순위 확장이 있습니다. arXiv 사전 인쇄본 arXiv:1405.3866, 2014.

마하일 코닥, 닐 테넨홀츠, 레스터 매키, 니콜로 푸시. 초기화와 정규화 요인화된 신경층, 2021.

디에데릭 P. 킹마, 지미 바. 아담: 확률적 최적화를 위한 방법, 2017.

Dmitry Lepikhin, HyounJoong Lee, Yuanzhong Xu, Dehao Chen, Orhan Firat, Yanping Huang, Maxim Krikun, Noam Shazeer, Zhifeng Chen. Gshard: 조건부 계산과 자동 샤딩을 이용한 거대 모델 확장, 2020.

브라이언 레스터, 라미 알-르푸, 노아 콘스탄트. 매개변수 효율적 프롬프트 튜닝을 위한 스케일의 힘. arXiv:2104.08691 [cs], 2021년 4월. URL <http://arxiv.org/abs/2104.08691>. arXiv: 2104.08691.

Chunyan Li, Heerad Farkhoor, Rosanne Liu 및 Jason Yosinski. 객관적인 풍경의 본질적인 차원을 측정합니다. arXiv:1804.08838 [cs, stat], 2018년 4월. URL <http://arxiv.org/abs/1804.08838>. arXiv: 1804.08838.

Xiang Lisa Li와 Percy Liang. 접두어 튜닝: 생성을 위한 연속 프롬프트 최적화. arXiv:2101.00190 [cs], 2021년 1월. URL <http://arxiv.org/abs/2101.00190>.

리 위안지(Yuanzhi Li)와 량 잉위(Yingyu Liang). 구조화된 데이터에 대한 확률적 경사 하강법을 통한 과매개변수화 신경망 학습. 신경 정보 처리 시스템의 발전, 2018.

리 위안지, 량 잉위, 안드레이 리스테스키. 교대 최소화를 통한 가중 저순위 근사의 회복 보장. 기계 학습 국제 학술 대회, pp.

2358–2367. PMLR, 2016.

리 위안지, 마 텐구, 장 홍양. 과매개변수화된 행렬 센싱 및 이차 활성화를 포함하는 신경망에서의 알고리즘적 정규화. 학습 이론 컨퍼런스, 2–47쪽. PMLR, 2018b.

Zhaojiang Lin, Andrea Madotto, Pascale Fung. 매개변수 효율적 전이 학습을 통한 다재다능한 생성 언어 모델 탐구. EMNLP 2020 연구 결과: 441–459쪽, 온라인, 2020년 11월. EMNLP. doi: 10.18653/v1/2020.findings-emnlp.41. URL <https://aclanthology.org/2020.findings-emnlp.41>.

Xiao Liu, Yanan Zheng, Zhengxiao Du, Ming Ding, Yujie Qian, Zhilin Yang 및 Jie Tang. GPT 이해합니다. arXiv:2103.10385 [cs], 2021년 3월. URL <http://arxiv.org/abs/2103.10385>. arXiv: 2103.10385.

Yinhan Liu, Myle Ott, 나만 고알, Jingfei Du, Mandar Joshi, Danqi Chen, Omer Levy, Mike Lewis, Luke Zettlemoyer, Veselin Stoyanov. Roberta: 견고하게 최적화된 bert 사전 학습 접근 방식, 2019.

Ilya Loshchilov와 Frank Hutter. 분리된 가중치 감소 정규화. arXiv 사전 인쇄본 arXiv:1711.05101, 2017.

일리아 로실로프와 프랭크 후터. 분리된 가중치 감소 정규화, 2019.

라비 카리미 마하바디, 제임스 헨더슨, 세바스찬 루더. 콤팩터: 효율적인 저등급 초복잡한 어댑터 계층, 2021.

Linyong Nan, Dragomir Radev, Rui Zhang, Amrit Rau, Abhinand Sivaprasad, Chiachun Hsieh, Xiangru Tang, Aadit Vyas, Neha Verma, Pranav Krishna 등 다트: 오픈 도메인 구조 데이터 레코드에서 텍스트 생성까지. arXiv 사전 인쇄본 arXiv:2007.02871, 2020.

에카테리나 노비코바, 온드레이 두섹, 베레나 리저. e2e 데이터 세트: 최종 목표를 위한 새로운 과제 최종 세대. arXiv 사전 인쇄본 arXiv:1706.09254, 2017.

Samet Oymak, Zalan Fabian, Mingchen Li, Mahdi Soltanolkotabi. 야코비안의 저랭크 구조를 활용한 신경망의 일반화 보장. arXiv 사전 인쇄본 arXiv:1906.05392, 2019.

조나스 파이퍼, 아이슈와라 카마스, 안드레아스 러클, 경현 조, 이리나 구레비치. 어댑터-융합: 전이 학습을 위한 비파괴적 과제 구성, 2021.

Daniel Povey, Gaofeng Cheng, Yiming Wang, Ke Li, Hainan Xu, Mahsa Yarmohammadi, San-jeev Khudanpur. 심층 신경망을 위한 반직교 저랭크 행렬 분해. 인터스피치, 3743-3747쪽, 2018.

알렉스 래드포드, 카르틱 나라심한, 팀 살리만스, 일리아 수츠케버. 생성적 사전 훈련을 통한 언어 이해 향상. 12쪽, a.

알렉스 래드포드, 제프리 우, 리원 차일드, 데이비드 루안, 다리오 아모데이, 일리아 수츠케버. 언어 모델은 비지도 멀티태스크 학습자입니다. pp. 24, b.

프리나브 라즈푸르카르, 로빈 지아, 퍼시 리안. 모르는 것을 아는 것: 답할 수 없는 질문들. 본대용. CoRR, abs/1806.03822, 2018. URL <http://arxiv.org/abs/1806.03822>.

실베스트르 알비제 레부피, 하칸 빌렌, 안드레아 베달디. 다양한 시각 영역 학습 잔류 어댑터. arXiv:1705.08045 [cs, stat], 2017년 11월. URL <http://arxiv.org/abs/1705.08045>. arXiv: 1705.08045.

Andreas Ruckl, Gregor Geigle, Max Glockner, Tilman Beck, Jonas Pfeiffer, Nils Reimers 및 이리나 구레비치. 어댑터드롭: 변압기 어댑터의 효율성에 관하여, 2020.

Tara N Sainath, Brian Kingsbury, Vikas Sindhwani, Ebru Arisoy, Bhuvana Ramabhadran. 고차원 출력 타겟을 갖는 심층 신경망 학습을 위한 저순위 행렬 분해. 2013년 IEEE 음향, 음성 및 신호 처리 국제 컨퍼런스, pp. 6655-6659. IEEE, 2013.

Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper 및 Bryan Catanzaro. 메가트론-lm: 모델 병렬화를 사용하여 수십억 개의 매개변수를 가진 언어 모델을 학습시키는 방법, 2020.

Richard Socher, Alex Perelygin, Jean Wu, Jason Chuang, Christopher D. Manning, Andrew Ng, 그리고 Christopher Potts. 감정에 대한 의미적 구성성을 위한 재귀적 심층 모델 나무 은닉. 2013년 자연어 경합적 방법론 학술대회 논문집. Processing, pp. 1631-1642, 미국 워싱턴주 제네라, 2013년 10월. 전산언어학회. URL <https://aclanthology.org/D13-1170>.

아사쉬 바스와니, 노암 샤지르, 니키 파마르, 야콥 우스코레이트, 리온 존스, 에이단 N. 고메즈, 루카스 카이저, 일리아 폴로수킨. 주의만 기울이면 됩니다. 제31회 신경 정보 처리 시스템 국제 학술대회 논문집, 6000-6010쪽, 2017.

알렉스 왕, 아만프리트 싱, 줄리안 마이클, 펠릭스 힐, 오머 레비, 사무엘 R. 보우먼.  
Glue: 자연어 이해를 위한 멀티태스킹 벤치마크 및 분석 플랫폼, 2019년.

알렉스 왕, 야다 프록사차쿰, 니키타 낭기아, 아만프리트 싱, 줄리안 마이클, 펠릭스 힐, 오메르 레비, 사무엘 R. 보우먼. 슈퍼글루: 범용 언어 이해 시스템을 위한 더욱 강력한 벤치마크, 2020.

알렉스 워슈타트, 아만프리트 싱, 사무엘 R 보우먼. 신경망 수용성 판단.  
arXiv 사전 인쇄 arXiv:1805.12471, 2018.

Adina Williams, Nikita Nangia, Samuel Bowman. 추론을 통한 문장 이해를 위한 광범위한 챌린지 코퍼스. 2018년 미국 전산언어학회 북미 지부 학술대회 논문집: 인간언어기술, 제1권(장편논문), 1112-1122쪽, 루이지애나주 뉴올리언스, 2018년 6월. 전산언어학회. doi: 10.18653/v1/N18-1101. URL <https://www.aclweb.org/anthology/N18-1101>.

토마스 울프, 리산드르 드브뤼트, 빅터 산, 줄리앙 쇼몽, 클레망 들랑그, 앤서니 무아, 피에릭 시스타크, 팀 로, 레미 루프, 모건 푼토비츠, 조 데이비스, 샘 솔라이퍼, 패트릭 폰 플라텐, 클라라 마, 야신 저니테, 줄리앙 플루, 칸웬 쉬, 테브 르 스카오, 실뱅 구거, 마리아마 드라마메, 켄틴 뢰스트, 알렉산더 M. 라시. 트랜스포머: 최첨단 자연어 처리. 2020 자연어 처리 경험적 방법 학회 논문집: 시스템 시연, 38-45쪽, 온라인, 2020년 10월. 전산언어학회. URL <https://www.aclweb.org/anthology/2020.emnlp-demos.6>.

그렉 양과 에드워드 J. 후. 무한 폭 신경망에서의 특징 학습. arXiv:2011.14522 [cond-mat], 2021년 5월. URL <http://arxiv.org/abs/2011.14522>. arXiv: 2011.14522.

엘라드 벤 자켄, 사울리 라브포겔, 요아브 골드버그. Bitfit: 간단한 매개변수 효율적인 미세 조정  
변압기 기반 마스크 언어 모델, 2021.

Yu Zhang, Ekapol Chuangsuwanich, James Glass. 저순위 행렬 분해를 이용한 심층 신경망 병목 현상 특징 추출. 2014 IEEE 음향, 음성 및 신호 처리 국제 학술대회(ICASSP), 185-189쪽. IEEE, 2014.

용 자오, 진위 리, 이판 공. 심층 신경망을 위한 저랭크 플러스 대각 적용.  
2016년 IEEE 음향, 음성 및 신호 처리 국제 컨퍼런스(ICASSP), 5005-5009쪽. IEEE, 2016.

Victor Zhong, Caiming Xiong, Richard Socher. Seq2sql: 강화 학습을 이용한 자연어 기반 구조적 쿼리 생성. CoRR, abs/1709.00103, 2017. URL <http://arxiv.org/abs/1709.00103>.

대규모 언어 모델에는 여전히 매개 변수 업데이트가 필요합니다.

소수의 학습 샘플만 있을 때는 퓨샷 학습(Few-shot learning) 또는 프롬프트 엔지니어링(prompt engineering)이 매우 유용합니다. 하지만 실제로는 성능에 민감한 애플리케이션을 위해 수천 개 이상의 학습 샘플을 큐레이션할 여유가 있는 경우가 많습니다. 표 8에서 볼 수 있듯이, 미세 조정은 크고 작은 데이터셋에서 퓨샷 학습에 비해 모델 성능을 크게 향상시킵니다. GPT-3 논문(Brown et al., 2020)에서 RTE에 대한 GPT-3 퓨샷 결과를 가져왔습니다. MNLI 매칭의 경우, 클래스당 두 개의 데모와 총 여섯 개의 컨텍스트 내 예제를 사용했습니다.

| 방법          | MNLI-m (평가액/%) RTE (평가액/%) |      |
|-------------|----------------------------|------|
| GPT-3 퓨샷    | 40.6                       | 69.0 |
| GPT-3 미세 조정 | 89.5                       | 85.4 |

표 8: 미세 조정은 GPT-3에서 few-shot learning보다 상당히 우수한 성능을 보입니다(Brown et al., 2020).

B 어댑터 계층에서 발생하는 추론 지연

어댑터 계층은 사전 학습된 모델에 순차적으로 추가되는 외부 모듈인 반면, 본 논문에서 제안하는 LoRA는 병렬적으로 추가되는 외부 모듈로 볼 수 있습니다. 따라서 어댑터 계층은 기본 모델 외에 추가로 계산되어야 하며, 이로 인해 불가피하게 추가적인 지연 시간이 발생합니다. Ruckliffe et al. (2020)에서 지적했듯이, 어댑터 계층으로 인해 발생하는 지연 시간은 모델 배치 크기 및/또는 시퀀스 길이가 하드웨어 병렬성을 최대한 활용할 수 있을 만큼 충분히 클 때 완화될 수 있습니다. 본 논문에서는 GPT-2 매체에서 유사한 지연 시간 연구를 통해 이러한 관찰 결과를 확인했으며, 특히 온라인 추론과 같이 배치 크기가 작은 경우에도 추가 지연 시간이 상당할 수 있는 시나리오가 있음을 지적합니다.

NVIDIA Quadro RTX8000에서 단일 순방향 패스의 지연 시간을 100회 이상의 시행 평균을 통해 측정했습니다. 입력 배치 크기, 시퀀스 길이, 그리고 어댑터 병목 현상 차원 r을 변경했습니다. 두 가지 어댑터 설계를 테스트했습니다. Houlshby 외(2019)가 제안한 최초의 설계(AdapterH)와 Lin 외(2020)가 제안한 최근의 더 효율적인 변형(AdapterL)입니다. 설계에 대한 자세한 내용은 5.1절을 참조하십시오. 그림 5는 어댑터가 없는 기준선과 비교한 속도 저하 비율을 백분율로 표시했습니다.



그림 5: 어댑터 없음( $r = 0$ ) 기준선과 비교한 추론 속도 저하 비율. 위쪽은 AdapterH에 대한 결과를, 아래쪽은 AdapterL에 대한 결과를 보여줍니다. 배치 크기와 시퀀스 길이가 더 길면 지연 시간을 줄이는 데 도움이 되지만, 온라인에서 시퀀스 길이가 짧은 시나리오에서는 속도 저하가 최대 30% 이상 발생할 수 있습니다. 가시성을 높이기 위해 컬러맵을 조정했습니다.

C 데이터 세트 세부 정보

GLUE 벤치마크는 광범위한 자연어 이해 과제들을 모아놓은 것입니다. 여기에는 MNLI(추론, Williams et al. (2018)), SST-2(감정 분석, Socher et al. (2013)), MRPC(의역 감지, Dolan & Brockett (2005)), CoLA(언어적 수용성, Warstadt et al.

(2018)), QNLI(추론, Rajpurkar et al.(2018)), QQP8 (질의응답), RTE(추론),

<sup>8</sup><https://quoradata.quora.com/First-Quora-데이터셋-릴리스-질문-쌍>

및 STS-B(텍스트 유사성, Cer et al. (2017)). GLUE 벤치마크는 광범위한 적용 범위를 가지고 있어 RoBERTa 및 DeBERTa와 같은 NLU 모델을 평가하는 표준 지표로 사용됩니다. 각 데이터셋은 서로 다른 허가 라이선스에 따라 배포됩니다.

WikiSQL은 Zhong et al. (2017)에서 소개되었으며, 56,355/8,421개의 학습/검증 예제를 포함하고 있습니다. 이 학습의 목적은 자연어 질문과 테이블 스키마로부터 SQL 쿼리를 생성하는 것입니다.

컨텍스트를  $x = \{\text{테이블 스키마, 쿼리}\}$ 로, 대상을  $y = \{\text{SQL}\}$ 로 인코딩합니다. 이 데이터셋은 BSD 3-Clause 라이선스에 따라 배포됩니다.

SAMSum은 Gliwa 외(2019)에서 소개되었으며, 14,732/819개의 훈련/테스트 예제를 포함하고 있습니다. 두 사람 간의 단계별 채팅 대화와 언어학자가 작성한 해당 요약으로 구성됩니다. 맥락은 " $\backslash n$ "으로 연결된 발화 다음에 " $\backslash n \backslash n$ "이 오는 형태로, 목표는  $y = \{\text{summary}\}$ 로 인코딩됩니다. 이 데이터셋은 비상업적 라이선스인 Creative Commons BY-NC-ND 4.0에 따라 배포됩니다.

E2E NLG 챌린지는 Novikova 외 연구진(2017)에서 종단 간 데이터 기반 자연어 생성 시스템 학습을 위한 데이터셋으로 처음 소개되었으며, 데이터-텍스트 변환 평가에 널리 사용됩니다. E2E 데이터셋은 레스토랑 도메인에서 약 42,000개의 학습 예제, 4,600개의 검증 예제, 그리고 4,600개의 테스트 예제로 구성됩니다. 입력으로 사용되는 각 소스 테이블은 여러 참조를 가질 수 있습니다. 각 샘플 입력( $x, y$ )은 슬롯-값 쌍의 시퀀스와 해당 자연어 참조 텍스트로 구성됩니다. 이 데이터셋은 크리에이티브 커먼즈 BY-NC-SA 4.0 라이선스에 따라 배포됩니다.

DART는 Nan et al. (2020)에서 설명된 오픈 도메인 데이터-텍스트 데이터셋입니다. DART 입력은 ENTITY — RELATION — ENTITY 트리플 시퀀스로 구성됩니다. 총 8만 2천 개의 예제를 포함하는 DART는 E2E에 비해 훨씬 더 크고 복잡한 데이터-텍스트 작업입니다. 이 데이터셋은 MIT 라이선스에 따라 배포됩니다.

WebNLG는 데이터-텍스트 평가에 일반적으로 사용되는 또 다른 데이터셋입니다(Gardent et al., 2017). 총 22,000개의 예시를 포함하는 WebNLG는 14개의 개별 범주로 구성되며, 그중 9개는 학습 과정에서 관찰됩니다. 총 14개 범주 중 5개는 학습 과정에서 관찰되지 않지만 테스트 세트에는 나타나므로, 평가는 일반적으로 "보인" 범주(S), "보이지 않은" 범주(U), 그리고 "전체" 범주(A)로 구분됩니다. 각 입력 예시는 SUBJECT — PROPERTY — OBJECT 트리플 시퀀스로 표현됩니다. 이 데이터셋은 크리에이티브 커먼즈 BY-NC-SA 4.0 라이선스에 따라 배포됩니다.

## 실험에 사용되는 D 하이퍼파라미터

### D.1 로베르타

선형 학습률 감소 스케줄을 사용하여 AdamW를 사용하여 학습합니다. LoRA에 대해 학습률, 학습 에폭 수, 배치 크기를 변경합니다. Liu et al. (2019)에 따라 MRPC, RTE, STS-B에 적용할 때 일반적인 초기화 대신 LoRA 모듈을 최적의 MNLI 체크포인트로 초기화합니다. 사전 학습된 모델은 모든 작업에 대해 고정 상태를 유지합니다. 5개의 랜덤 시드에 대한 중앙값을 보고합니다. 각 실행의 결과는 최상의 에폭에서 가져옵니다. Houlsby et al. (2019) 및 Pfeiffer et al. (2021)의 설정과 공정하게 비교하기 위해 모델 시퀀스 길이를 128로 제한하고 모든 작업에 고정된 배치 크기를 사용했습니다. 중요한 점은 MRPC, RTE, STS-B에 적용할 때 이미 MNLI에 적용된 모델 대신 사전 학습된 RoBERTa 대형 모델로 시작한다는 것입니다.

이 제한된 설정으로 실행한 결과는  $\uparrow$ 로 표시되어 있습니다. 실험에 사용된 하이퍼파라미터는 표 9에서 확인할 수 있습니다.

### D.2 데베르타

선형 학습률 감소 스케줄을 적용하여 AdamW를 사용하여 다시 학습합니다. He et al. (2021)에 따라 학습률, 드롭아웃 확률, 워밍업 단계, 그리고 배치 크기를 조정합니다. 공정한 비교를 위해 He et al. (2021)에서 사용한 것과 동일한 모델 시퀀스 길이를 사용합니다. He et al. (2021)에 따라 MRPC, RTE, STS-B에 적용할 때 일반적인 초기화 대신 최적의 MNLI 체크포인트로 LoRA 모듈을 초기화합니다. 사전 학습된 모델은 모든 작업에 대해 고정됩니다. 5개의 랜덤 시드에 대한 중앙값을 보고하며, 각 실행의 결과는 최적의 에폭에서 가져옵니다. 실험에 사용된 하이퍼파라미터는 표 10을 참조하십시오.

| 방법                        | 데이터 세트                                                                        | MNLI SST-2 MRPC CoLA QNLI QQP RTE STS-B |      |      |     |     |     |      |     |
|---------------------------|-------------------------------------------------------------------------------|-----------------------------------------|------|------|-----|-----|-----|------|-----|
|                           | 최적화기<br>워밍업 비율<br>LR 일정                                                       | 아담W<br>0.06<br>선익                       |      |      |     |     |     |      |     |
| 로베르타 기지<br>로라             | 배치 크기 16 16 # 에포크 30 25 학습 속도 5E-04 5E-04 4E-04 4E-04 4E-04 4E-04 4E-04 4E-04 | 32                                      | 16   |      |     |     |     |      |     |
|                           | 4E-04                                                                         | 60                                      | 30   | 80   | 25  |     |     | 80   | 40  |
|                           | LoRA 구성.<br>LoRA $\alpha$ 최<br>대 시퀀스 길이                                       | rq = rv = 8<br>8<br>512                 |      |      |     |     |     |      |     |
| 로베르타 라지<br>로라             | 배치 크기 4 4 # 에포크 10 20 학습 속도 3E-04 4E-04 3E-04 2E-04 2E-04 3E-04 4E-04         |                                         |      |      |     |     |     |      | 8   |
|                           | 2E-04                                                                         | 10                                      | 4 20 | 4 20 | 10  |     |     | 8 20 | 30  |
|                           | LoRA 구성.<br>LoRA $\alpha$ 최<br>대 시퀀스 길이                                       | 128                                     | 128  | 512  | 128 | 512 | 512 | 512  | 512 |
| 로베르타 라지<br>로라 †           | 배치 크기 # 에<br>포크 10 20 학습 속도 3E-04 4E-04 3E-04 2E-04 2E-04 3E-04 4E-04         |                                         |      |      |     |     |     |      | 4   |
|                           | 10 20 학습 속도 3E-04 4E-04 3E-04 2E-04 2E-04 3E-04 4E-04                         | 20                                      | 10   |      |     |     |     |      | 10  |
|                           | LoRA 구성.<br>LoRA $\alpha$ 최<br>대 시퀀스 길이                                       | rq = rv = 8<br>16<br>128                |      |      |     |     |     |      |     |
| 로베르타 라지<br>AdptP (3M) †   | 배치 크기 #<br>Epoch 10 20 학습 속도 3E-05 3E-05 3E-04 3E-04 3E-04 3E-04 3E-04        |                                         |      |      |     |     |     |      | 32  |
|                           | 20 학습 속도 3E-05 3E-05 3E-04 3E-04 3E-04 3E-04 3E-04                            | 20                                      | 20   | 20   | 10  |     |     | 20   | 20  |
|                           | 병목 현상 r 최대<br>시퀀스 길이                                                          | 64<br>128                               |      |      |     |     |     |      |     |
| 로베르타 라지<br>AdptP (0.8M) † | 배치 크기 #<br>Epoch 5 20 학습 속도 3E-04 3E-04 3E-04 3E-04 3E-04 3E-04 3E-04         |                                         |      |      |     |     |     |      | 32  |
|                           | 20 학습 속도 3E-04 3E-04 3E-04 3E-04 3E-04 3E-04 3E-04                            | 20                                      | 20   | 20   | 10  |     |     | 20   | 20  |
|                           | 병목 현상 r 최대<br>시퀀스 길이                                                          | 16<br>128                               |      |      |     |     |     |      |     |
| 로베르타 라지<br>AdptH (6M) †   | 배치 크기 #<br>Epoch 10 20 학습 속도 3E-05 3E-04 3E-04 3E-04 3E-04 3E-04 3E-04        |                                         |      |      |     |     |     |      | 32  |
|                           | 20 학습 속도 3E-05 3E-04 3E-04 3E-04 3E-04 3E-04 3E-04                            | 5                                       |      |      |     |     |     | 20   | 10  |
|                           | 병목 현상 r 최대<br>시퀀스 길이                                                          | 64<br>128                               |      |      |     |     |     |      |     |
| 로베르타 라지<br>AdptH (0.8M) † | 배치 크기 #<br>Epoch 10 20 학습 속도 3E-04 3E-04 3E-04 3E-04 3E-04 3E-04 3E-04        |                                         |      |      |     |     |     |      | 32  |
|                           | 20 학습 속도 3E-04 3E-04 3E-04 3E-04 3E-04 3E-04 3E-04                            | 5                                       |      |      |     |     |     | 20   | 10  |
|                           | 병목 현상 r 최대<br>시퀀스 길이                                                          | 8<br>128                                |      |      |     |     |     |      |     |

표 9: GLUE 벤치마크에서 RoBERTa에 사용한 하이퍼파라미터.

D.3 GPT-2

우리는 AdamW(Loshchilov & Hutter, 2017)를 사용하여 선형 학습을 통해 모든 GPT-2 모델을 훈련합니다. 5개 에포크에 대한 속도 스케줄. 앞서 설명한 배치 크기, 학습률, 빔 탐색 빔 크기를 사용합니다. Li & Liang (2021)에서. 따라서 LoRA에 대한 위의 하이퍼파라미터도 조정합니다. 3개의 난수 시드에 대한 평균입니다. 각 실행에 대한 결과는 가장 좋은 에포크에서 가져옵니다. 하이퍼파라미터 GPT-2의 LoRA에 사용된 기준은 표 11에 나와 있습니다. 다른 기준에 사용된 기준은 Li & Liang을 참조하세요. (2021).

D.4 GPT-3

모든 GPT-3 실험의 경우, 우리는 AdamW(Loshchilov & Hutter, 2017)를 사용하여 2개의 에포크 동안 훈련합니다. 배치 크기는 128개 샘플이고 가중치 감소 계수는 0.1입니다. 시퀀스 길이는 384입니다.

| 방법             | 데이터 세트                                                                               | MNLI SST-2 MRPC CoLA QNLI QQP RTE STS-B |
|----------------|--------------------------------------------------------------------------------------|-----------------------------------------|
|                | 최적화기<br>위밍업 비율<br>LR 일정                                                              | 아담W<br>0.1<br>선의                        |
| 데베르타 XXL<br>로라 | 배치 크기 8 8 # 에포크 5 11 학습률 1E-04 6E-05 2E-05 3E-05 1E-04 1E-04 1E-04 2E-04 4E-04 2E-04 | 4 4<br>11 10                            |
|                | 가중치 감소 0 0.01 CLS 드롭아웃 0.15 0.2 LoRA 구성 0.01 0.01 0.1 0.1<br>rq = rv = 8             | 0.01 0.1<br>0.2 0.2                     |
|                | LoRA α 최                                                                             | 8                                       |
|                | 대 시퀀스 길이 256                                                                         | 128 128 64 512 320 320 128              |

표 10: GLUE 벤치마크에 포함된 작업에 대한 DeBERTa XXL의 하이퍼매개변수.

| 데이터 세트      | E2E 웹NLG DART |
|-------------|---------------|
|             | 훈련            |
| 최적화기        | 아담W           |
| 체중 감소       | 0.01 0.01 0.0 |
| 중도 탈락 가능성   | 0.1 0.1 0.0   |
| 배치 크기       | 8             |
| # 시대        | 5             |
| 위밍업 단계      | 500           |
| 학습률 일정      | 선의            |
| 라벨 부드럽게     | 0.1 0.1 0.0   |
| 학습률         | 0.0002        |
| 적응          | rq = rv = 4   |
| 로라 알파       | 32            |
|             | 추론            |
| 빔 크기        | 10            |
| 길이 페널티 반복 없 | 0.9 0.8 0.8   |
| 음 ngram 크기  | 4             |

표 11: E2E, WebNLG 및 DART의 GPT-2 LoRA에 대한 하이퍼매개변수.

WikiSQL(Zhong et al., 2017), MNLI(Williams et al., 2018)의 경우 768, SAMSum(Gliwa)의 경우 2048 et al., 2019). 모든 방법-데이터셋 조합에 대해 학습률을 조정합니다. 자세한 내용은 D.4절을 참조하십시오. 사용된 하이퍼파라미터에 대한 세부 정보. 접두사 임베딩 튜닝을 위해 최적의 lp와 li를 찾습니다. 각각 256과 8이 되어 총 3.2M개의 학습 가능한 매개변수가 됩니다. lp = 8, li = 8을 사용합니다. 20.2M개의 학습 가능한 매개변수를 사용하여 접두사-계층 튜닝을 수행하여 전체적으로 최상의 성능을 얻었습니다. LoRA에 대한 두 가지 매개변수 예산을 제시합니다: 4.7M(rq = rv = 1 또는 rv = 2) 및 37.7M(rq = rv = 8) 또는 rq = rk = rv = ro = 2). 각 실행에서 가장 좋은 검증 성능을 보고합니다. 훈련 GPT-3 실험에 사용된 하이퍼매개변수는 표 12에 나열되어 있습니다.

LORA와 PREFIX TUNEING을 결합한 E

LoRA는 기존의 접두사 기반 접근 방식과 자연스럽게 결합될 수 있습니다. 이 섹션에서는 WikiSQL과 MNLI에서 LoRA와 접두사 튜닝의 두 가지 조합. LoRA+PrefixEmbed(LoRA+PE)는 LoRA와 접두사 임베딩 튜닝을 결합합니다. lp + li 특수 토큰은 임베딩이 학습 가능한 매개변수로 처리됩니다. 접두사 임베딩 튜닝에 대한 자세한 내용은 5.1절을 참조하십시오. LoRA+PrefixLayer(LoRA+PL)는 LoRA와 Prefix-Layer 튜닝을 결합합니다. 또한 lp + li를 삽입합니다. 특수 토큰; 그러나 이러한 토큰의 숨겨진 표현이 자연스럽게 진화하도록 두는 대신

| 하이퍼파라미터 PreEmbed PreLayer BitFit AdapterH LoRA 미세 조정 |          |          |          |                           |
|------------------------------------------------------|----------|----------|----------|---------------------------|
| 최적화기                                                 | 아담W      |          |          |                           |
| 배치 크기                                                | 128      |          |          |                           |
| # 시대                                                 | 2        |          |          |                           |
| 워밍업 토큰                                               | 25만      |          |          |                           |
| LR 일정                                                | 선의       |          |          |                           |
| 학습률                                                  | 5.00E-06 | 5.00E-04 | 1.00E-04 | 1.6E-03 1.00E-04 2.00E-04 |

표 12: 다양한 GPT-3 적응 방법에 사용된 훈련 하이퍼파라미터.  
학습률을 조정한 후 모든 데이터 세트에 동일한 하이퍼파라미터를 적용했습니다.

렐리에서 우리는 모든 Transformer 블록 이후에 입력에 독립적인 벡터로 대체합니다. 따라서 임베딩 및 후속 Transformer 블록 활성화는 학습 가능한 매개변수로 처리됩니다. 접두사 계층 튜닝에 대한 자세한 내용은 섹션 5.1을 참조하세요.

표 15에서는 WikiSQL과 MultiNL에서 LoRA+PE와 LoRA+PL의 평가 결과를 보여줍니다. 우선, LoRA+PE는 LoRA와 접두사 임베딩 튜닝보다 훨씬 더 우수한 성능을 보입니다. WikiSQL은 LoRA가 접두사 임베딩 튜닝과 어느 정도 직교적임을 나타냅니다. LoRA+PE의 조합인 MultiNL은 LoRA보다 성능이 더 좋지 않습니다. 아마도 LoRA가 그 자체로 이미 인간 기준선과 비슷한 성능을 달성합니다. 둘째, 우리는 LoRA+PL은 더 많은 학습 가능 매개변수를 사용하더라도 LoRA보다 성능이 약간 떨어집니다. 이는 접두사 계층 튜닝이 학습률 선택에 매우 민감하기 때문에 발생하는 것으로 생각됩니다. LoRA+PL에서 LoRA 가중치 최적화를 더 어렵게 만듭니다.

F 추가 경험적 실험

F.1 GPT-2 에 대한 추가 실험

또한 DART(Nan et al., 2020) 및 WebNLG(Gardent et al., 2017)에 대한 실험을 반복합니다. Li & Liang (2021)의 설정에 따라 계산되었습니다. 결과는 표 13에 나와 있습니다. 본 연구의 결과와 유사합니다. 섹션 5에 보고된 E2E NLG Challenge에서 LoRA는 최소한 동등하거나 더 나은 성능을 보였습니다. 동일한 수의 학습 가능한 매개변수가 주어졌을 때 접두사 기반 접근 방식입니다.

| 방법               | # 훈련 가능     | 다트      |       |      |
|------------------|-------------|---------|-------|------|
|                  | 매개변수 BLEU ↑ | MET ↑   | TER ↓ |      |
| GPT-2 매체         |             |         |       |      |
| 미세 조정 354M 0.37M |             | 46.2    | 0.39  | 0.46 |
| 어댑터L             |             | 42.4    | 0.36  | 0.48 |
| 어댑터L             | 11개월        | 45.2    | 0.38  | 0.46 |
| FT 톱2            | 24개월        | 41.0    | 0.34  | 0.56 |
| 프리레이어            | 0.35개월      | 46.4    | 0.38  | 0.46 |
| 로라               | 0.35개월      | 47.1±.2 | 0.39  | 0.46 |
| GPT-2 대형         |             |         |       |      |
| 미세 조정 774M       |             | 47.0    | 0.39  | 0.46 |
| 어댑터L             | 0.88M       | 45.7±.1 | 0.38  | 0.46 |
| 어댑터L             | 23M         | 47.1±.1 | 0.39  | 0.45 |
| 프리레이어            | 0.77M       | 46.7    | 0.38  | 0.45 |
| 로라               | 0.77M       | 47.5±.1 | 0.39  | 0.45 |

표 13: DART에서 다양한 적응 방법을 적용한 GPT-2. MET와 TER의 분산은 다음과 같습니다. 모든 적응 접근 방식에 대해 0.01 미만입니다.

| 방법                          | WebNLG       |              |                         |                         |
|-----------------------------|--------------|--------------|-------------------------|-------------------------|
|                             | 유            | 블루 ↑<br>메스   | 호주미국<br>메트 ↑            | 터 ↓                     |
| GPT-2 매체                    |              |              |                         |                         |
| Fine-Tune(354M) 27.7 어댑터L   |              | 64.2         | 46.5 .30                | 45 .38 .76 .33 .53      |
| (0.37M) 45.1 어댑터L (11M)     |              | 54.5         | 50.2 .36                | 39 .38 .46 .40 .43      |
|                             | 48.3         | 60.4 .38 .43 | 54.9 .35 .39            |                         |
| FTTop2 (24M)                | 18.9         | 53.6 .23 .38 | 36.0 .99 .49 .72        |                         |
| 접두사(0.35M)                  | 45.6         | 62.9 .38 .44 | 55.1 .49 .35 .40        |                         |
| 로라(0.35M)                   | 46.7 ± .4    | 62.1 ± .2    | 55.3 ± .2               | .38 .44 .41 .46 .33 .39 |
| GPT-2 대형                    |              |              |                         |                         |
| Fine-Tune(774M) 43.1 어댑터L   |              | 65.3         | 55.5 .38                | 46 .42 .53 .33 .42      |
| (0.88M) 49.8 ± .0           | 61.1 ± .0    | 56.0 ± .0    | .38 .43 .41 .44 .35 .39 |                         |
| 어댑터L (23M) 49.2 ± .1        | 64.7 ± .2    | 57.7 ± .1    | .39 .46 .43 .46 .33 .39 |                         |
| 접두사(0.77M) 63.4 .39 .45 .42 | 48.7 .34 .40 | 56.3         |                         |                         |
| LoRA (0.77M) 48.4 ± .3      | 64.0 ± .3    | 57.0 ± .1    | .39 .45 .42 .45 .32 .38 |                         |

표 14: WebNLG에서 다양한 적응 방식을 적용한 GPT-2. MET와 TER의 분산  
우리가 실행한 모든 실험에서 0.01보다 작습니다. "U"는 보이지 않는 범주를 나타내고 "S"는 보이는 범주를 나타냅니다.  
카테고리, 그리고 "A"는 WebNLG 테스트 세트의 모든 카테고리를 나타냅니다.

F.2 GPT-3 에 대한 추가 실험

우리는 표 15에서 다른 적응 방법을 사용한 GPT-3에 대한 추가 실행을 제시합니다. 초점은 다음과 같습니다.  
성능과 훈련 가능한 매개변수 수 간의 균형을 파악합니다.

F.3 저데이터 체제

저데이터 체제에서 다양한 적응 접근 방식의 성능을 평가하기 위해 우리는 무작위로  
MNLI의 전체 학습 세트에서 100, 1k 및 10k 학습 예제 샘플을 추출하여 저데이터를 형성합니다.  
MNLI-n 작업. 표 16에서는 MNLI-n에 대한 다양한 적응 접근법의 성능을 보여줍니다. 놀랍게도 PrefixEmbed와 PrefixLayer는  
MNLI-100 데이터셋에서 매우 낮은 성능을 보였습니다.  
PrefixEmbed는 random chance보다 약간 더 나은 성능을 보였습니다(37.6% 대 33.3%). PrefixLayer  
PrefixEmbed보다 성능이 우수하지만 MNLI-100에서 Fine-Tune이나 LoRA보다 여전히 현저히 떨어집니다. 학습 예제 수가 증가함에  
따라 접두사 기반 접근법과 LoRA/Fine-tuning 간의 격차가 줄어들어 접두사 기반 접근법이 적합하지 않음을 시사할 수 있습니다.

GPT-3의 저데이터 작업에 적합합니다. LoRA는 두 가지 모두에서 미세 조정보다 더 나은 성능을 달성합니다.  
MNLI-100 및 MNLI-Full, 그리고 MNLI-1k 및 MNLI-10K에 대한 비교 가능한 결과  
(±0.3) 무작위 씨앗으로 인한 분산.

MNLI-n에 대한 다양한 적응 접근 방식의 훈련 하이퍼 매개변수는 표 17에 보고되어 있습니다. 훈련 손실이 발생하기 때문에 MNLI-100  
세트의 PrefixLayer에 대해 더 작은 학습률을 사용합니다.  
학습률이 높아져도 감소하지 않습니다.

G 부분 공간 간 유사성 측정

본 논문에서는  $\phi(A, B, i, j) = \psi(U)$  측정을 사용합니다.

$$U_{ij} = \frac{\sum_{k=1}^n \sum_{l=1}^m \phi(A_k, B_l, i, j)}{\sum_{k=1}^n \sum_{l=1}^m \phi(A_k, B_l, i, j)}$$

두 열 직교 행렬 U 간의 유사성  
A와 B의 왼쪽 특이 행렬의 열을 취합니다. 우리는 이 유사성이 단순히 다음과 같다는 점을 지적합니다.  
Ham & Lee 부분 공간 간 거리를 측정하는 표준 투영 메트릭의 역  
(2008).

| 방법      | 하이퍼파라미터                               | # 학습 가능한 매개변수 | WikiSQL | MNLI-m |
|---------|---------------------------------------|---------------|---------|--------|
| 미세 조정   | -                                     | 175B          | 73.8    | 89.5   |
| 접두사 삽입  | lp = 32, li = 8                       | 0.4M          | 55.9    | 84.9   |
|         | lp = 64, li = 8                       | 0.9M          | 58.7    | 88.1   |
|         | lp = 128, li = 8                      | 1.7M          | 60.6    | 88.0   |
|         | lp = 256, li = 8                      | 3.2M          | 63.1    | 88.6   |
|         | lp = 512, li = 8                      | 6.4M          | 55.9    | 85.8   |
| 접두사 레이어 | lp = 0, li = 8                        | 5.1M          | 68.5    | 89.2   |
|         | lp = 32, li = 4                       | 10.1M         | 69.8    | 88.2   |
|         | lp = 64, li = 0                       | 20.2M         | 70.1    | 89.5   |
|         |                                       | 44.1M         | 66.4    | 89.6   |
|         |                                       | 76.1M         | 64.9    | 87.9   |
| 어댑터H    | r = 1                                 | 7.1M          | 71.9    | 89.8   |
|         | r = 4                                 | 21.2M         | 73.2    | 91.0   |
|         | r = 8                                 | 40.1M         | 73.2    | 91.5   |
|         | r = 16                                | 77.9M         | 73.2    | 91.5   |
|         | r = 64                                | 304.4M        | 72.6    | 91.5   |
| 로라      | rv = 2                                | 4.7백만         | 73.4    | 91.7   |
|         | rq = rv = 1                           | 4.7백만         | 73.4    | 91.3   |
|         | rv = 2, rq = rk                       | 9.4M          | 73.3    | 91.4   |
|         | rv = ro = 1, rq = rv = 4, rk =        | 9.4M          | 74.1    | 91.2   |
|         | rv = ro = 2, rk = rv = ro = 2         | 18.8M         | 73.7    | 91.3   |
|         | rq = rv = 8, rk = rv = ro =           | 18.8M         | 73.7    | 91.7   |
|         | 4, rk = rv = 64                       | 37.7M         | 73.8    | 91.6   |
|         | rq = rk = rv = ro = 64, rk = rv       | 37.7M         | 74.0    | 91.7   |
|         | = 8, lp = 8, li = 4                   | 301.9M        | 73.6    | 91.4   |
|         | rq = rv = 32, lp = 8, li = 4, rk =    | 603.8M        | 73.9    | 91.4   |
| 로라+PE   | rv = 64, lp = 8, li = 4, rk = rv = 8, | 37.8M         | 75.0    | 91.4   |
|         | lp = 8, li = 4                        | 151.1M        | 75.9    | 91.1   |
|         |                                       | 302.1M        | 76.2    | 91.3   |
| 로라+PL   |                                       | 52.8백만        | 72.9    | 90.2   |

표 15: WikiSQL과 MNLI의 다양한 적용 방식에 대한 하이퍼파라미터 분석. 접두사 임베딩 튜닝(PrefixEmbed) 및 접두사 레이어 튜닝(PrefixLayer)은 다음과 같이 성능이 저하됩니다. 학습 가능한 매개변수 수를 늘리는 반면 LoRA의 성능은 안정화됩니다. 성능은 검증 정확도로 측정됨.

| 방법             | MNLI(m)-100 | MNLI(m)-1k | MNLI(m)-10k | MNLI(m)-392K |
|----------------|-------------|------------|-------------|--------------|
| GPT-3(미세 조정)   | 60.2        | 85.8       | 88.9        | 89.5         |
| GPT-3(프릭스 임베드) | 37.6        | 75.2       | 79.5        | 88.6         |
| GPT-3(접두사 계층)  | 48.3        | 82.5       | 85.9        | 89.6         |
| GPT-3(로라)      | 63.8        | 85.6       | 89.2        | 91.7         |

표 16: GPT-3 175B를 사용하여 MNLI 하위 집합에 대한 다양한 방법의 검증 정확도. MNLI-n은 n개의 학습 예제를 갖는 하위 집합을 나타냅니다. 전체 검증 집합을 사용하여 평가합니다. LoRA 미세 조정을 포함한 다른 방법과 비교했을 때 샘플 효율성이 더 우수합니다.

구체적으로, U의 단위 값을 다음과 같이 두십시오.  $\sigma_1, \sigma_2, \dots$ 가 됩니다.  $\sigma_p$  여기서  $p = \min\{i, j\}$ . 우리는 투영 지표 Ham & Lee(2008)는 다음과 같이 정의됩니다.

$$D(U_i, U_j) = \sqrt{\sum_{p=1}^{\min(i,j)} \sigma_p^2} \in [0, \sqrt{p}]$$

| 하이퍼파라미터 | 적응         | MNLI-100 | MNLI-1k  | MNLI-10K    | MNLI-392K |
|---------|------------|----------|----------|-------------|-----------|
| 최적화기    | -          |          |          | 아담W         |           |
| 워밍업 토큰  | -          |          |          | 25만         |           |
| LR 일정   | -          |          |          | 선의          |           |
| 배치 크기   | -          | 20       | 20       | 100         | 128       |
| # 시대    | -          | 40       | 40       | 4           | 2         |
| 학습률     | 파인튜닝       |          |          | 5.00E-6     |           |
|         | 접두사 삽입     | 2.00E-04 | 2.00E-04 | 4.00E-04    | 5.00E-05  |
|         | 접두사 레이어    | 5.00E-05 | 5.00E-05 | 2.00E-4     |           |
|         | 로라         |          |          |             | 5.00E-04  |
| 적응-특정한  | 접두사 임베드 lp | 16       | 32       | 64          | 256       |
|         | 접두사임베드 li  |          |          | 8           |           |
|         | 프릭스톤       |          |          | lp = li = 8 |           |
|         | 로라         |          |          | rq = rv = 8 |           |

표 17: MNLI(m)-n에 대한 다양한 GPT-3 적응 방법에 사용된 하이퍼 매개변수.

여기서 우리의 유사성은 다음과 같이 정의됩니다.

$$\phi(A, B, i, j) = \psi(U_{\text{에이}}, U_{\text{B}}) = \frac{\sum_{i=1}^n \sum_{j=1}^n U_{\text{에이}} U_{\text{B}}}{\sum_{i=1}^n \sum_{j=1}^n U_{\text{에이}}^2} = \frac{1}{\sum_{i=1}^n \sum_{j=1}^n U_{\text{에이}}^2} \sum_{i=1}^n \sum_{j=1}^n U_{\text{에이}} U_{\text{B}}$$

이 유사성은 U가 다음과 같은 경우를 만족합니다. 그리고  $\phi(A, B, i, j) = 1$ 입니다. 완전히 직교하는 경우  $\phi(A, B, i, j) = 0$ 입니다. 그렇지 않은 경우  $\phi(A, B, i, j) \in (0, 1)$ 입니다.

H 저랭크 행렬 에 대한 추가 실험

우리는 저랭크 업데이트 행렬에 대한 조사에서 얻은 추가 결과를 제시합니다.

H.1 LORA 모듈 간 상관 관계

그림 3과 그림 4에 제시된 결과가 다른 것에 어떻게 일반화되는지 알아보려면 그림 6과 그림 7을 참조하세요.   
충.

H.2 r 이 GPT-2 에 미치는 영향

우리는 GPT-2에서 r의 효과에 대한 실험을 반복합니다(섹션 7.2). E2E NLG 챌린지를 활용합니다. 예를 들어 데이터 세트를 사용하여 다양한 선택에 의해 달성된 검증 손실 및 테스트 메트릭을 보고합니다. 26,000단계 학습 후 r의 최적 순위. 표 18에 결과를 제시합니다. GPT-2의 최적 순위 중간값은 사용된 지표에 따라 4~16 사이이며, 이는 GPT-3 175B의 중간값과 유사합니다. 모델 크기와 적응을 위한 최적 순위 간의 관계는 여전히 열려 있습니다.   
질문.

H.3 W 와 ΔW 의 상관관계

r이 변할 때 W와 ΔW 사이의 정규화된 부분 공간 유사도는 그림 8을 참조하세요.

다시 한번, ΔW에는 W의 상위 특이 방향이 포함되지 않는다는 점에 유의하십시오. ΔW의 상위 4개 방향과 W의 상위 10% 방향은 0.2를 간신히 넘습니다. 이는 증거를 제공합니다. ΔW에는 W에서 강조되지 않는 "작업별" 지침이 포함되어 있습니다.

다음으로 답해야 할 흥미로운 질문은 우리가 작업별 확장을 얼마나 "강력하게" 해야 하는가입니다. 모델 적응이 잘 작동하려면 어떤 지침을 따라야 합니까?

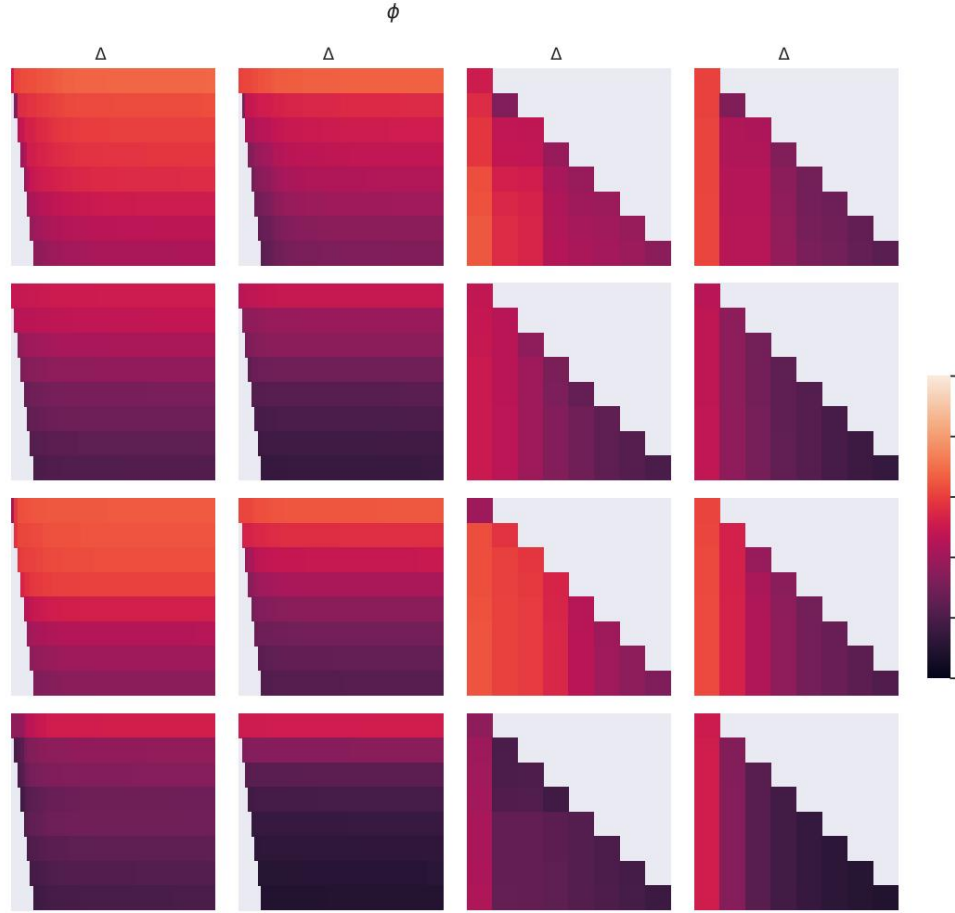


그림 6: 96층 변압기의 1, 32, 64, 96층에서  $\Delta W_q$  와  $\Delta W_v$ 에 대한  $Ar=8$  과  $Ar=64$  의 열 벡터 간의 정규화된 부분 공간 유사성 .

#### H.4 증폭 계수

$\Delta W$ 의 SVD 분해에서 좌특이 행렬과 우특이 행렬의 비율인 특징 증폭 계수를 자연스럽게 고려  $\frac{\Delta W_F}{UW_{대 F}}$  , 여기서  $U$ 와  $V$  할 수 있습니다. ( $UU^T WVV^T$ 는  $W$ 를  $\Delta W$ 로 생성되는 부분 공간에 "투영"한 것을 나타냅니다.)

직관적으로,  $\Delta W$ 가 주로 작업별 방향을 포함할 때, 이 양은  $\Delta W$ 에 의해 얼마나 증폭되는지를 측정합니다. 7.3절에서 볼 수 있듯이,  $r = 4$ 의 경우 이 증폭 계수는 최대 20입니다. 다시 말해, (일반적으로) 각 계층에는 (사전 학습된 모델  $W$ 의 전체 특징 공간 중에서) 4개의 특징 방향이 있으며, 이는 다운스트림 특정 작업에 대해 보고된 정확도를 달성하기 위해 매우 큰 계수인 20만큼 증폭되어야 합니다. 그리고 각 다운스트림 작업에 대해 매우 다른 특징 방향 세트가 증폭될 것으로 예상해야 합니다.

그러나  $r = 64$ 의 경우, 이 증폭 계수는 약 2에 불과합니다. 이는  $r = 64$ 인  $\Delta W$ 에서 학습된 대부분의 방향이 크게 증폭되지 않음을 의미합니다. 이는 놀라운 일이 아니며, 실제로 "작업별 방향"(따라서 모델 적응을 위해)을 나타내는 데 필요한 내재적 순위가 낮다는 것을 (다시 한번) 증명합니다. 이와 대조적으로,  $\Delta W$ 의 순위 4 버전( $r = 4$ 에 해당)의 방향은 훨씬 더 큰 20배만큼 증폭됩니다.

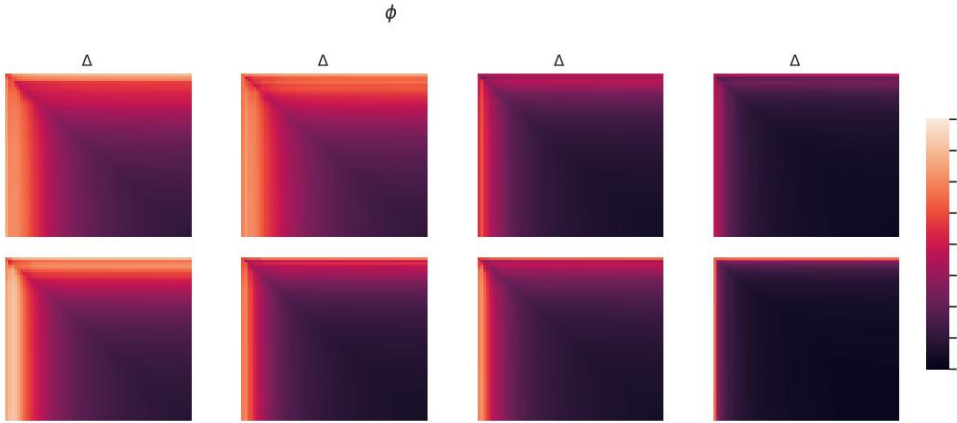


그림 7: 두 개의 무작위 Ar=64 열 벡터 간의 정규화된 부분 공간 유사성  
96층 변압기의 1, 32, 64, 96층에서 ΔWq 와 ΔWv에 대한 시드 런이 실행되었습니다.

| 랭크 r 가치 | 손실   | BLEU   | NIST   | METEOR | ROUGE L | CIDEr  | -      |
|---------|------|--------|--------|--------|---------|--------|--------|
| 1       | 1.23 | 68.72  | 8.7215 | 0.4565 | 69.17   | 0.7052 | 2.4329 |
| 2       | 1.21 | 8.7413 | 0.4590 | 70.38  | 8.8439  | 0.7052 | 2.4639 |
| 4       | 1.18 | 8.1.17 | 0.4689 | 69.57  | 8.7457  | 0.4636 | 0.7186 |
| 16      | 1.16 | 32     | 69.61  | 8.7483 | 0.4629  | 69.33  | 0.7196 |
| 1.16    | 64   | 1.16   | 8.7736 | 0.4642 | 69.24   | 8.7174 | 0.7177 |
| 128     | 1.16 | 256    | 0.4651 | 68.73  | 8.6718  | 0.4628 | 0.7105 |
| 1.16    | 512  | 1.16   | 68.92  | 8.6982 | 0.4629  | 68.78  | 0.7180 |
| 1024    | 1.17 |        | 8.6857 | 0.4637 | 69.37   | 8.7495 | 0.7127 |
|         |      |        | 0.4659 |        |         |        | 0.7128 |
|         |      |        |        |        |         |        | 0.7128 |
|         |      |        |        |        |         |        | 0.7149 |

표 18: LoRA가 달성한 E2E NLG 챌린지의 검증 손실 및 테스트 세트 메트릭  
GPT-2 Medium을 사용하여 다른 순위 r을 계산합니다. r = 10이면 많은 작업에 충분한 GPT-3와 달리, 여기서는  
검증 손실의 경우 r = 16에서 성능이 최고조에 달하고 BLEU의 경우 r = 4에서 성능이 최고조에 달하여 GPT-2가  
Medium은 GPT-3 175B와 비교했을 때 적응에 대한 내재적 순위가 비슷합니다.  
하이퍼 매개변수는 다른 기준의 매개변수 수와 일치하는 r = 4에서 조정됩니다.  
따라서 다른 r 선택에는 최적이지 아닐 수 있습니다.

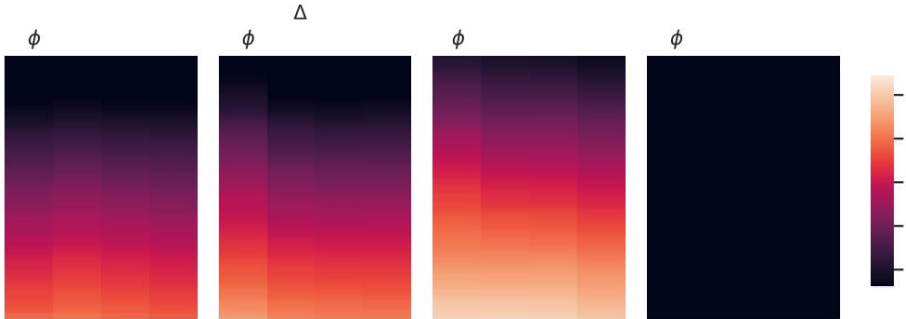


그림 8: Wq의 특이 방향과 ΔWq의 특이 방향 간의 정규화된 부분 공간 유사성  
r이 변하고 무작위 기준선이 있는 경우 ΔWq는 W에서 중요하지만 강조되지 않는 방향을 증폭합니다. r이 더 큰 ΔW는 이미 강조된 방  
향을 더 많이 선택하는 경향이 있습니다.  
더블유.